

Project Report

Universidad Peruana de Ciencias Aplicadas



Ingeniería de Software - 2025-20
Aplicaciones para Dispositivos Móviles - 1798
Profesor: Jorge Luis Mayta Guillermo

Informe del Trabajo Final

Startup: ATG
Producto: Autonexo

Team Members:

Member	Code
Cruz Ibarra, Victor Andres	u202311053
Iglesias Pérez, Sergio Sebastián	u202316118
Roman Esteban, Henry Kalet	u202310210
Solano Armas, Angelo Hector	u20231B775
Vivanco Salazar, Rafael Andres	u202311064

Septiembre, 2025

Registro de Versiones del Informe

Versión	Fecha	Autor	Descripción de modificación
TB1	07/09/2025	Todos los integrantes del grupo aportaron	Reporte, capítulos 1 y 2

Project Report Collaboration Insights

Enlace de la organización del proyecto: <https://github.com/ATG-UPC>

Para el desarrollo del informe correspondiente a la entrega TB1, se repartieron las tareas de la siguiente manera para cada integrante del equipo:

Integrante	Tareas Asignadas
Victor Andres Cruz Ibarra	Descripción de la Startup, Perfiles de integrantes del equipo, Lean UX Problem Statements, Lean UX Assumptions, User Stories, Impact Mapping, Product Backlog, Application Layer, Infrastructure Layer
Rafael Andres Vivanco Salazar	Antecedentes y problemática, Lean UX Hypothesis Statements, Lean UX Canvas, EventStorming, Candidate Context Discovery, Domain Message Flows Modeling, Bounded Context Software Architecture Component Level Diagrams, Software Architecture Code Level Diagrams
Sergio Sebastián Iglesias Pérez	Segmentos objetivo, Análisis competitivo, Estrategias frente a competidores, Bounded Context Canvases, Context Mapping, Software Architecture Context Level Diagrams, Software Architecture Container Level Diagrams
Henry Kalet Esteban Roman	Diseño de entrevistas, Registro de entrevistas, Análisis de entrevistas, Software Architecture Deployment Diagrams
Angelo Hector Solano Armas	Needfinding, User Personas, User Task Matrix, User Journey Mapping, Empathy Mapping, Ubiquitous Language, Bounded Context: , Domain Layer, Interface Layer

Contenido

Tabla de contenidos

- **Contenido**
 - [Tabla de contenidos](#)
- **Student Outcome**
- **Capítulo I: Presentación**
 - **1.1. Startup Profile**
 - **1.1.1. Descripción de la Startup**
 - **1.1.2. Perfiles de integrantes del equipo**
 - **1.2. Solution Profile**
 - **1.2.1. Antecedentes y problemática**
 - **1.2.2. Lean UX Process**
 - **1.2.2.1. Lean UX Problem Statements**
 - **1.2.2.2. Lean UX Assumptions**
 - **1.2.2.3. Lean UX Hypothesis Statements**
 - **1.2.2.4. Lean UX Canvas**
 - **1.3. Segmentos objetivo**
- **Capítulo II: Requirements Development and Software Solution Design**
 - **2.1. Competidores**
 - **2.1.1. Análisis competitivo**
 - **2.1.2. Estrategias y tácticas frente a competidores**

- 2.2. Entrevistas
 - 2.2.1. Diseño de entrevistas
 - 2.2.2. Registro de entrevistas
 - 2.2.3. Análisis de entrevistas
- 2.3. Needfinding
 - 2.3.1. User Personas
 - 2.3.2. User Task Matrix
 - 2.3.3. User Journey Mapping
 - 2.3.4. Empathy Mapping
 - 2.3.5. Ubiquitous Language
- 2.4. Requirements specification
 - 2.4.1 User Stories
 - 2.4.2 Impact Mapping
 - 2.4.3 Product Backlog
- 2.5. Strategic-Level Domain-Driven Design
 - 2.5.1. EventStorming
 - 2.5.1.1 Candidate Context Discovery
 - 2.5.1.2 Domain Message Flows Modeling
 - 2.5.1.3 Bounded Context Canvases
 - 2.5.2. Context Mapping
 - 2.5.3. Software Architecture
 - 2.5.3.1 Software Architecture Context Level Diagrams
 - 2.5.3.2 Software Architecture Container Level Diagrams
 - 2.5.3.3 Software Architecture Deployment Diagrams
- 2.6 Tactical-Level Domain-Driven Design
 - 2.6.1 Bounded Context: Bounded Context Name
 - 2.6.1.1 Domain Layer
 - 2.6.1.2 Interface Layer
 - 2.6.1.3 Application Layer
 - 2.6.1.4 Infrastructure Layer
 - 2.6.1.5 Bounded Context Software Architecture Component Level Diagrams
 - 2.6.1.6 Bounded Context Software Architecture Code Level Diagrams
 - 2.6.1.6.1 Bounded Context Domain Layer Class Diagrams
 - 2.6.1.6.2 Bounded Context Database Design Diagram
- Capítulo III: Solution UI/UX Design
 - 3.1. Product design
 - 3.1.1. Style Guidelines
 - 3.1.1.1 General Style Guidelines
 - 3.1.2. Information Architecture
 - 3.1.2.1 Organization Systems
 - 3.1.2.2 Labeling Systems
 - 3.1.2.3 SEO Tags and Meta Tags
 - 3.1.2.4. Searching Systems
 - 3.1.2.5. Navigation Systems
 - 3.1.3. Landing Page UI Design
 - 3.1.3.1. Landing Page Wireframe
 - 3.1.3.2. Landing Page Mock-up
 - 3.1.4. Mobile Applications UX/UI Design
 - 3.1.4.1. Mobile Applications Wireframes
 - 3.1.4.2. Mobile Applications Wireflow Diagrams
 - 3.1.4.3. Mobile Applications Mock-ups
 - 3.1.4.4. Mobile Applications User Flow Diagram
 - 3.1.4.5. Mobile Applications Prototyping

- **Capítulo IV: Product Implementation & Validation**
 - **4.1. Software Configuration Management**
 - **4.1.1. Software Development Environment Configuration**
 - **4.1.2. Source Code Management**
 - **4.1.3. Source Code Style Guide & Conventions**
 - **4.1.4. Software Deployment Configuration**
 - **4.2. Landing Page, Services & Mobile Implementation**
 - **4.2.1. Sprint 1**
 - **4.2.1.1. Sprint Planning 1**
 - **4.2.1.2. Sprint Backlog 1**
 - **4.2.1.3. Development Evidence for Sprint Review**
 - **4.2.1.4. Testing Suite Evidence for Sprint Review**
 - **4.2.1.5. Execution Evidence for Sprint Review**
 - **4.2.1.6. Services Documentation Evidence for Sprint Review**
 - **4.2.1.7. Software Deployment Evidence for Sprint Review**
 - **4.2.1.8. Team Collaboration Insights during Sprint**
 - **4.3. Validation Interviews**
 - **4.3.1. Diseño de Entrevistas**
 - **4.3.2. Registro de Entrevistas.**
 - **4.3.3. Evaluaciones según heurísticas**
- **Conclusiones**
 - **Conclusiones y recomendaciones**
 - **Video App-Validation**
 - **Video About-the-Product**
 - **Video About-the-Team**
- **Glosario**
- **Bibliografía**
- **Anexos**

Student Outcome

El curso contribuye al cumplimiento del Student Outcome ABET:
ABET – EAC - Student Outcome 7 Criterio: La capacidad de adquirir y aplicar nuevos conocimientos según sea necesario, utilizando estrategias de aprendizaje apropiadas.

Criterio específico	Acciones realizadas	Conclusiones
---------------------	---------------------	--------------

Criterio específico	Acciones realizadas	Conclusiones
Actualiza conceptos y conocimientos necesarios para su desarrollo profesional y en especial para su proyecto en soluciones de ingeniería de software	Andres Cruz: TB1: Al desarrollar la descripción de la startup y estructurar el Product Backlog con herramientas como Lean UX e Impact Mapping, actualicé mis conocimientos en metodologías ágiles y arquitectura de software, lo cual fortalece mis capacidades para aplicar soluciones en proyectos reales de ingeniería de software. Angelo Solano: TB1: Al realizar Needfinding, User Personas, Journey Maps y modelar la capa de dominio, actualicé mis conocimientos sobre experiencia de usuario y diseño de software en capas, lo que me permitió integrar la visión del usuario con la solución técnica en el proyecto. Henry Esteban: TB1: Mediante el diseño, registro y análisis de entrevistas, y la construcción del Deployment Diagram, reforcé mis conocimientos sobre técnicas de recolección de requerimientos y despliegue de software, actualizando competencias claves para el desarrollo de proyectos de ingeniería. Rafael Vivanco: TB1: Al analizar los segmentos objetivos, la competencia y mapear los contextos, actualicé mis conocimientos en análisis estratégico y en arquitectura de software a nivel de contenedores y contexto, lo que me permitió integrar tanto la visión de negocio como la técnica en el proyecto. Sergio Iglesias TB1: Al analizar los segmentos objetivos, la competencia y mapear los contextos, actualicé mis conocimientos en análisis estratégico y en arquitectura de software a nivel de contenedores y contexto, lo que me permitió integrar tanto la visión de negocio como la técnica en el proyecto.	TB1: Como grupo, actualizamos y fortalecimos nuestros conocimientos en metodologías ágiles, diseño de experiencia de usuario y arquitectura de software en diferentes niveles. Cada integrante aplicó conceptos actualizados que permitieron enriquecer el proyecto y avanzar en una solución más profesional y estructurada.

Criterio específico	Acciones realizadas	Conclusiones
Reconoce la necesidad del aprendizaje permanente para el desempeño profesional y el desarrollo de proyectos en soluciones de tecnologías de ingeniería de software.	Andres Cruz: TB1: Comprendí que siempre debo seguir aprendiendo sobre nuevas metodologías ágiles, backlog prioritization y herramientas de arquitectura, ya que estas prácticas evolucionan constantemente y son esenciales para responder a las demandas cambiantes de la industria. Angelo Solano: TB1: Trabajar en la definición de usuarios, journey maps y en la capa de interfaz me permitió reconocer que el aprendizaje permanente es fundamental para mantenerme actualizado en metodologías de diseño centrado en el usuario y en tecnologías de desarrollo de software. Henry Esteban: TB1: Durante la preparación y análisis de entrevistas reconocí que el aprendizaje permanente es clave para perfeccionar técnicas de levantamiento de información y adaptación de arquitecturas de despliegue a entornos modernos de software. Rafael Vivanco: TB1: Al aplicar técnicas avanzadas de modelado de dominios, reconocí la importancia del aprendizaje continuo para mantenerme actualizado en arquitecturas modernas y metodologías de diseño que impactan directamente en la calidad de los proyectos de software. Sergio Iglesias TB1: Al investigar sobre context mapping y estrategias frente a competidores, entendí que el aprendizaje permanente es necesario para responder a escenarios competitivos y tecnológicos que cambian rápidamente en la industria del software.	TB1: Reconocimos como equipo la importancia del aprendizaje permanente en ingeniería de software. Las herramientas, metodologías y técnicas que aplicamos en esta entrega (desde entrevistas y journey maps hasta diagramas arquitectónicos y event storming) nos mostraron que el conocimiento en este campo evoluciona constantemente, y que debemos mantenernos en actualización continua para garantizar soluciones de calidad en proyectos reales.

Capítulo I: Introducción

1.1. Startup Profile

1.1.1. Descripción de la Startup

Autonexo es una aplicación diseñada para conectar a los propietarios de vehículos con mecánicos especializados en todo tipo de mantenimiento vehicular, ya sea preventivo, correctivo o cualquier otro tipo de servicio automotriz. La aplicación actúa como un intermediario eficiente, proporcionando una solución integral que facilita la interacción entre los conductores y los profesionales del sector automotriz.

La plataforma permite a los propietarios acceder a un catálogo de mecánicos certificados y servicios disponibles, con la posibilidad de elegir en función de la especialización, reputación y proximidad del servicio. Por otro lado, los mecánicos tienen la oportunidad de promocionar sus servicios, gestionar su agenda de forma eficiente y recibir solicitudes de mantenimiento en tiempo real, lo que les permite maximizar su tiempo y la eficiencia de sus operaciones.

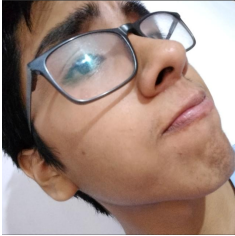
Autonexo se basa en la necesidad creciente de digitalización dentro de la industria automotriz, un sector que, según un estudio de Innocar en colaboración con la empresa mexicana Roshfrans, muestra una baja adopción de tecnologías especializadas en la gestión de talleres mecánicos (solo el 23.5% de los talleres en América Latina utilizan software especializado para gestionar sus operaciones). Este panorama resalta la urgencia de una solución digital que facilite el proceso de mantenimiento vehicular y promueva la eficiencia operativa, tanto para los mecánicos como para los propietarios de los vehículos.

La plataforma no solo mejora la accesibilidad al servicio automotriz, sino que también permite un registro detallado del historial de mantenimiento de cada vehículo, optimizando la gestión preventiva y reduciendo los costos a largo plazo para los conductores. Además, la capacidad de recibir atención personalizada y el acceso a mecánicos especializados en diversas áreas del mantenimiento incrementan la confianza y la satisfacción del usuario.

El objetivo de Autonexo es transformar la experiencia de mantenimiento vehicular, ofreciendo una solución accesible, transparente y efectiva tanto para los conductores como para los mecánicos. Al mismo tiempo, fomenta la adopción de la tecnología en un sector que históricamente ha dependido de métodos tradicionales y manuales, mejorando la eficiencia y el control sobre los costos de mantenimiento.

1.1.2. Perfiles de integrantes del equipo

	Angelo Solano - u20231B775 Mi nombre es Angelo Solano, soy estudiante de Ingeniería de Software en la UPC. Me apasiona la tecnología y todo lo relacionado con el desarrollo de software. Me gusta enfrentarme a desafíos complejos y encontrar soluciones creativas. Estoy en constante aprendizaje, siempre buscando mejorar mis habilidades en programación y análisis de sistemas. Me considero una persona comprometida con mis proyectos y con ganas de crecer tanto profesionalmente como personalmente. Disfruto trabajar en equipo y siempre trato de aportar lo mejor de mí en todo lo que hago.
	Henry Esteban - u202310210 Mi nombre es Henry Esteban, tengo 20 años y estoy cursando mi 6to ciclo de la carrera de Ingeniería de Software en la UPC. Soy una persona dedicada, analítica y con gran interés en el desarrollo de soluciones tecnológicas innovadoras. Me caracterizo por mi capacidad de trabajo en equipo y mi compromiso con la excelencia en cada proyecto que realizo. Estoy enfocado en aprender constantemente y aplicar mis conocimientos para crear aplicaciones que generen un impacto positivo en la sociedad.

	Sergio Iglesias - u202316118 Mi nombre es Sergio Iglesias, tengo 19 años y estoy cursando mi 6to ciclo de la carrera de Ingeniería de Software en la UPC. Soy una persona proactiva, creativa y con gran pasión por la tecnología. Me destaco por mi capacidad de resolver problemas de manera eficiente y mi habilidad para trabajar colaborativamente en proyectos complejos. Estoy comprometido con el aprendizaje continuo y siempre busco aplicar las mejores prácticas en el desarrollo de software. Mi objetivo es contribuir significativamente al éxito de este proyecto y crecer profesionalmente en el campo de la ingeniería de software.
	Victor Cruz - u202311053 Mi nombre es Victor Cruz, tengo 19 años y estoy cursando mi 6to ciclo de la carrera de Ingeniería de Software en la UPC. Soy una persona entusiasta, creativa y comprometida con cada actividad que realizo. Estoy decidido a dar lo mejor de mí en este proyecto para lograr resultados de calidad.
	Rafael Vivanco - u202311064 Mi nombre es Rafael Vivanco, tengo 19 años y actualmente curso el 6to ciclo de la carrera de Ingeniería de Software en la UPC. Me considero una persona responsable con facilidad para adaptarme a distintos entornos y manejar varias tareas a la vez. Siempre doy lo mejor de mí en cada proyecto, aportando dedicación, esfuerzo y actitud positiva.

1.2. Solution Profile

1.2.1. Antecedentes y problemática

Who (¿Quién?)

Afecta principalmente a **mecánicos y propietarios de vehículos**, quienes necesitan gestionar y acceder a servicios de mantenimiento de manera eficiente.

What (¿Qué?)

Actualmente, los mecánicos enfrentan dificultades para organizar, planificar y dar seguimiento a los mantenimientos de los vehículos. La ausencia de un registro centralizado y estandarizado provoca errores en el control de historial, retrasos en los servicios y decisiones poco informadas. Por su parte, los propietarios tienen dificultad para encontrar mecánicos confiables y servicios adecuados a sus necesidades, lo que genera desconfianza y pérdida de tiempo.

Where (¿Dónde?)

Esta problemática se observa en **talleres mecánicos tradicionales** y entre propietarios de vehículos que buscan servicios automotrices confiables, especialmente en entornos urbanos donde la demanda de mantenimiento es alta.

When (¿Cuándo?)

El problema es constante y se intensifica en periodos críticos, como fechas de mantenimiento preventivo recomendado, estaciones de alto uso del vehículo y ante situaciones donde un vehículo requiere reparación inmediata.

How (¿Cómo?)

Autonexo propone una solución mediante una **aplicación móvil** que conecta directamente a propietarios con mecánicos certificados. La app permite registrar vehículos, historial de mantenimiento y servicios solicitados, al mismo tiempo que los mecánicos pueden gestionar sus agendas y responder a solicitudes en tiempo real, optimizando tiempos y recursos.

How Much (¿Cuánto?)

Según estimaciones del sector, la ineficiencia en la gestión de mantenimiento puede incrementar los costos hasta en un **20%**. Con Autonexo se proyecta una **mejora del 30% al 50% en eficiencia operativa**, una **reducción de hasta 25% en gastos por mantenimientos correctivos** y un **incremento del 40% en trazabilidad y control del historial vehicular**, mejorando la experiencia tanto del mecánico como del propietario.

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statements

Autonexo tiene como objetivo ofrecer a talleres mecánicos y propietarios de vehículos una **solución digital integral** que permita centralizar los procesos clave del mantenimiento vehicular, incluyendo el registro de unidades, la planificación de mantenimientos, el control de repuestos, el cálculo de presupuestos y el seguimiento histórico de intervenciones.

Actualmente, la mayoría de talleres y propietarios gestionan el mantenimiento de forma **manual, fragmentada y desorganizada**. Muchos dependen de hojas de Excel, cuadernos o herramientas genéricas que no están adaptadas a las necesidades específicas del sector automotriz. Esto provoca **fallas en el control de registros históricos, mayores costos por mantenimientos correctivos, retrasos en la atención de servicios y decisiones poco informadas** respecto al cuidado de los vehículos.

La falta de estandarización y centralización **limita la eficiencia de los talleres**, reduce la confianza de los propietarios y afecta la calidad del servicio. Además, genera **gastos imprevistos y tiempos muertos** tanto para mecánicos como para propietarios. La ausencia de un sistema unificado impide llevar un seguimiento transparente y ordenado del estado de cada vehículo, impactando negativamente en la productividad de los técnicos y en la satisfacción de los clientes.

Pregunta clave de diseño:

¿Cómo podríamos centralizar y digitalizar la gestión del mantenimiento vehicular de forma escalable y accesible, permitiendo a mecánicos y propietarios **ahorrar tiempo, reducir costos, mejorar la trazabilidad y asegurar un mantenimiento constante sin complicaciones**?

1.2.2.2. Lean UX Assumptions

Business Assumptions

1. Nuestros clientes necesitan una plataforma digital centralizada que les permita gestionar de manera integral el mantenimiento de sus vehículos y talleres, ya que actualmente dependen de métodos manuales que generan retrasos y errores, y buscan optimizar la eficiencia y control de sus operaciones.
2. Estas necesidades se pueden resolver mediante un software especializado que integre procesos clave como el registro de vehículos, historial de mantenimiento, planificación de servicios preventivos y correctivos, control de repuestos, cálculo de presupuestos y seguimiento de intervenciones en tiempo real, permitiendo centralizar toda la información y mejorar la trazabilidad de los servicios.
3. Los clientes iniciales serán propietarios de vehículos particulares y mecánicos independientes o de talleres pequeños que actualmente utilizan hojas de Excel, cuadernos o aplicaciones genéricas, y que requieren un sistema confiable para organizar sus mantenimientos y brindar un mejor servicio.
4. El valor más importante que buscan nuestros clientes es un control confiable, eficiente y accesible del mantenimiento vehicular y de los servicios del taller, reduciendo errores, retrasos y gastos innecesarios, mientras aumentan la confianza y satisfacción de los usuarios.

5. Adicionalmente, los usuarios pueden obtener beneficios tangibles como ahorro de tiempo y dinero en mantenimientos correctivos y preventivos, presupuestos claros y detallados para cada servicio, así como acceso a una red de mecánicos o propietarios disponibles en tiempo real según proximidad, especialización y reputación.
6. La adquisición de clientes se realizará mediante marketing digital en redes sociales como Facebook, Instagram y TikTok, campañas en Google Ads enfocadas en búsquedas de servicios automotrices, programas de referidos que incentiven la recomendación de la app, así como alianzas estratégicas con talleres automotrices para fomentar la adopción temprana.
7. El modelo de monetización será a través de suscripción mensual para mecánicos y talleres, otorgándoles acceso a herramientas avanzadas de gestión del taller, listado de propietarios y vehículos registrados en la plataforma, funcionalidades de planificación, presupuesto y seguimiento de historial.
8. La competencia principal incluye aplicaciones como Drivvo, Fleetio y otros softwares genéricos de gestión vehicular, pero nuestro diferencial radica en una interfaz amigable, localización precisa y funcionalidades adaptadas tanto para talleres como para propietarios, incluyendo búsqueda de mecánicos en tiempo real, historial completo y trazable de cada vehículo, y agenda optimizada para talleres.
9. Los venceremos gracias a nuestro enfoque local, la facilidad de uso de la plataforma y las funcionalidades específicas diseñadas para satisfacer tanto a propietarios individuales como a técnicos mecánicos, proporcionando eficiencia operativa y trazabilidad completa del mantenimiento vehicular.
10. El mayor riesgo es que los usuarios no adopten la plataforma debido a la preferencia por métodos tradicionales o a la falta de familiaridad con herramientas digitales, lo que podría limitar el crecimiento y uso del sistema.
11. Para mitigar este riesgo, se implementará una interfaz simple e intuitiva, acompañada de capacitaciones básicas y materiales de apoyo para mecánicos y propietarios, demostraciones gratuitas y pruebas piloto, así como testimonios de clientes satisfechos que incentiven la confianza y la adopción de la plataforma, asegurando que Autonexo cumpla con su objetivo de centralizar y digitalizar la gestión del mantenimiento vehicular de manera eficiente, accesible y confiable.

User Assumptions

1. ¿Quién es el usuario?

Nuestros usuarios son principalmente **propietarios de vehículos particulares** que buscan cuidar su inversión y optimizar los costos de mantenimiento, así como **mecánicos independientes o de talleres pequeños** que desean llevar un control digital eficiente de múltiples mantenimientos y agendas de servicio, ya que actualmente dependen de métodos manuales o herramientas poco integradas.

2. ¿Qué problema tiene nuestro producto que debe resolver?

El problema principal que Autonexo debe resolver es la **desorganización y fragmentación en la gestión del mantenimiento vehicular**, que genera sobrecostos, retrasos, fallas mecánicas evitables y pérdida de tiempo en tareas repetitivas o manuales, afectando tanto la eficiencia de los talleres como la satisfacción de los propietarios.

3. ¿Qué características son importantes?

Para cubrir estas necesidades, el producto debe ofrecer **registro integral de vehículos y usuarios**, una **plataforma de búsqueda de mecánico en tiempo real**, **cálculo de presupuestos de mantenimientos**, **historial detallado de intervenciones por vehículo** y **servicio de mensajería directa entre usuario y mecánico**, todo accesible desde dispositivos móviles para gestionar mantenimientos de manera centralizada y eficiente.

4. ¿Dónde encaja nuestro producto en su trabajo o vida?

El producto encaja directamente en el **día a día de los propietarios y mecánicos**: los propietarios lo utilizan para planificar, registrar y dar seguimiento a los mantenimientos de sus vehículos, mientras que los mecánicos gestionan múltiples unidades y presupuestos desde un solo lugar, optimizando su tiempo y reduciendo errores operativos.

5. ¿Cuándo y cómo es usado nuestro producto?

Autonexo será utilizado **cada vez que un usuario necesite planificar, registrar o dar seguimiento a un mantenimiento**, accediendo principalmente desde dispositivos móviles para garantizar flexibilidad, disponibilidad inmediata y control en tiempo real, sin importar si se encuentran en el taller, en ruta o en casa.

6. ¿Cómo debe verse nuestro producto y cómo debe comportarse?

El diseño y comportamiento del producto deben reflejar un **diseño moderno, limpio y amigable**, con menús intuitivos y fáciles de navegar, garantizando que el sistema responda rápidamente, evite interrupciones y proporcione una **experiencia fluida y confiable**, apta tanto para usuarios con experiencia digital como para aquellos que prefieren soluciones sencillas y accesibles.

1.2.2.3. Lean UX Hypothesis Statements

Hypothesis 01

Creemos que al ofrecer una plataforma centralizada para registrar vehículos, usuarios y mantenimientos, ayudaremos a los usuarios a organizar sus procesos de mantenimiento de manera más eficiente y reducir errores o pérdidas de información.

Sabremos que hemos tenido éxito

Cuando al menos el 70% de los usuarios registre y mantenga actualizado el historial de sus vehículos y mantenimientos durante el primer mes de uso.

Hypothesis 02

Creemos que al integrar herramientas de registro de mantenimiento y cálculo automático de presupuestos, facilitaremos la gestión operativa y financiera de los mecánicos y propietarios de vehículos, permitiéndoles planificar y ejecutar mantenimientos con mayor precisión.

Sabremos que hemos tenido éxito

Cuando al menos el 60% de los usuarios utilicen estas funciones para planificar o ejecutar mantenimientos dentro de la plataforma.

Hypothesis 03

Creemos que al diseñar una interfaz móvil intuitiva, clara y accesible, incentivaremos el uso constante de Autonexo incluso por usuarios con poca experiencia digital.

Sabremos que hemos tenido éxito

Cuando al menos el 75% de los usuarios activos utilicen la plataforma semanalmente para gestionar sus vehículos.

Hypothesis 04

Creemos que al permitir el acceso al historial completo de mantenimiento de cada vehículo, incrementaremos la confianza de los usuarios en sus decisiones de reparación, mantenimiento preventivo o venta de vehículos.

Sabremos que hemos tenido éxito

Cuando al menos el 50% de los usuarios consulten el historial como parte del proceso de evaluación del estado de sus vehículos.

1.2.2.4. Lean UX Canvas

FALTA COLOCAR EL MIRROR (LA IMAGEN Y ABAJO EL ENLACE QUE TE MANDE AHI)

1.3. Segmentos objetivo

Con el propósito de llegar de manera efectiva a posibles clientes, Autonexo ha definido dos segmentos principales como público objetivo.

Segmento objetivo #1: Propietarios

Personas que poseen uno o más vehículos personales y desean gestionar de forma eficiente el mantenimiento, control de gastos y estado general de su unidad, evitando olvidos o problemas mecánicos por falta de seguimiento.

Aspectos demográficos:

Sexo: Masculino y femenino,

Rango de edad: 25–50 años,
Nivel socioeconómico: Clases B y C (media-alta y media).

Aspectos geográficos:

Nacionalidad: Perú,
Zona geográfica: Urbana (principalmente Lima Metropolitana y otras ciudades con alta concentración vehicular).

Aspectos psicográficos:

Intereses: Cuidado del vehículo, control financiero, tecnología práctica, seguridad vial, soluciones digitales simples,
Estilo de vida: Conducen regularmente, valoran la comodidad y buscan evitar gastos imprevistos o pérdidas de tiempo por fallas mecánicas,
Actitudes: Son conscientes de la importancia del mantenimiento vehicular y están abiertos a herramientas digitales que les faciliten llevar un control ordenado y accesible desde su celular.

Necesidades clave:

Recordatorios de mantenimiento, historial de mantenimiento accesible, planificación de servicios, reducción de costos por mantenimientos imprevistos.

Comportamiento digital:

Uso de apps móviles, notificaciones y alertas, búsqueda de información y servicios de manera digital y rápida.

Segmento objetivo #2: Mecánicos

Profesionales que trabajan en talleres y desean gestionar mejor sus servicios y acceder a historiales de mantenimiento para brindar un mejor servicio a sus clientes.

Aspectos demográficos:

Sexo: Masculino (en su mayoría),
Rango de edad: 20–50 años,
Nivel socioeconómico: Clases C y D (media y media-baja).

Aspectos geográficos:

Nacionalidad: Perú,
Zona geográfica: Urbana (distritos con concentración de talleres y servicios automotrices).

Aspectos psicográficos:

Intereses: Reparación automotriz, optimización de tiempo, atención al cliente, soluciones digitales simples,
Estilo de vida: Profesionales prácticos, con un enfoque técnico, acostumbrados al trabajo manual, pero abiertos a soluciones tecnológicas si son simples y funcionales,
Actitudes: Desean mejorar la calidad de su servicio y organización interna, valoran plataformas que les permitan brindar un servicio más profesional sin complicaciones adicionales.

Necesidades clave:

Gestión eficiente de mantenimientos, acceso rápido a historiales de vehículos, reducción de errores en la planificación de servicios, optimización del tiempo de trabajo.

Comportamiento digital:

Familiaridad básica con smartphones, uso de aplicaciones simples y rápidas, disposición a digitalizar procesos si la herramienta es intuitiva y confiable.

Capítulo II: Requirements Development and Software Solution Design

2.1. Competidores

2.1.1. Análisis competitivo

Competitive Analysis Landscape					
¿Por qué llevar a cabo este análisis?		El objetivo es definir el posicionamiento de Autonexo frente a soluciones de gestión de flotas y mantenimiento vehicular, identificando diferenciadores de valor y riesgos competitivos para el mercado local.			
		AutoNexo  Fleetio Logo	Fleetio  Drivvo Logo	Drivvo  Drivvo Logo	Whip Around  Whip Around Logo
Perfil	Overview	Autonexo es una aplicación móvil que conecta propietarios de vehículos con mecánicos especializados, facilitando la gestión integral del mantenimiento vehicular mediante un sistema centralizado que incluye registro de vehículos, historial de mantenimientos, búsqueda de servicios y comunicación directa entre usuarios.	Fleetio es una plataforma SaaS integral de gestión de flotas empresariales que ofrece herramientas avanzadas para el mantenimiento preventivo, seguimiento de combustible, gestión de activos, inspecciones digitales y análisis de datos para optimizar operaciones de flotas comerciales.	Drivvo es una aplicación móvil orientada a conductores particulares y pequeñas empresas que permite registrar gastos de vehículo, programar recordatorios de mantenimiento, controlar consumo de combustible y generar reportes básicos de costos vehiculares de forma simple e intuitiva.	Whip Around es una plataforma especializada en inspecciones digitales y mantenimiento para flotas comerciales, enfocada en cumplimiento regulatorio, inspecciones DVIR personalizables, gestión de órdenes de reparación y reporting para compliance con normas de seguridad vehicular.
	Ventaja competitiva – ¿Qué valor ofrece al cliente?	Digitaliza y automatiza el proceso de mantenimiento; conecta demanda (conductores) con oferta (mecánicos) y da trazabilidad del historial.	Gestión centralizada y basada en datos para reducir costos (mantenimiento, combustible) y mejorar decisiones; módulos de inspección, órdenes de trabajo y fuel tracking.	Accesibilidad y simplicidad móvil para controlar gastos y recordatorios de servicio; apta para usuarios individuales.	Cumplimiento y seguridad: inspecciones digitalizadas y flujos mecánico–conductor para atender defectos rápido y cumplir normas (CSA/DOT).
Perfil de marketing	Mercado objetivo	Dueños de flotas o conductores individuales y mecánicos/talleres.	Gestores de flota en transporte, construcción, servicios y sector público.	Conductores y pequeños negocios que desean controlar gastos/consumo del vehículo.	Gerentes de flota, supervisores de operaciones y conductores en flotas comerciales.

Competitive Analysis Landscape					
	Estrategias de marketing	Performance en redes, programa de referidos y alianzas con talleres.	Contenido y recursos para fleet managers (guías, plantillas, blog), demos y pruebas gratuitas.	Web/app store pages + educación ligera (contenido de uso, ventajas).	Campañas orientadas a cumplimiento y reducción de riesgos; trial/demo.
Perfil de producto	Productos & servicios	Registro de vehículos/usuarios, planificación y seguimiento de mantenimientos, búsqueda de mecánico, mensajería y presupuestos.	Gestión de activos, mantenimiento preventivo/WO, inspecciones, combustible, partes/inventario, reportes y mobile app Fleetio Go.	Registro de gastos, combustible, recordatorios de servicio, reportes; opciones de Fleet para múltiples vehículos.	Inspecciones DVIR personalizables, mantenimiento, órdenes de reparación y reporting para compliance.
	Precios & costos	Suscripción por plan (accesible) para talleres y mecánicos; conductor gratuito o incluido (definición interna).	Suscripción mensual por plan (Essential/Professional) con usuarios ilimitados; add-ons por herramienta/activo; precios varían por tamaño.	Gratis (básico) + in-app purchases para Pro; en App Store figuran opciones mensuales/anuales y tiers de flota.	Suscripción por plan; mínimo mensual publicado: USD \$120 (Standard) y \$220 (Pro) en mes a mes; opciones "FixedUnlimited" para activos ilimitados.
	Canales de distribución	Web y app móvil.	Web + app móvil; usuarios ilimitados incluidos.	App móvil (iOS/Android); web principalmente informativa.	Web y app móvil.
Análisis SWOT	Fortalezas	Enfoque local, matching conductor-mecánico, trazabilidad del historial.	Cobertura funcional amplia y ecosistema maduro; fuerte orientación a datos.	Barrera de entrada baja y usabilidad móvil para usuario individual.	Especialista en inspecciones y cumplimiento; valor claro para safety/compliance.
	Debilidades	Requiere onboarding/educación para talleres no digitalizados.	Coste total puede crecer con activos/add-ons en flotas grandes.	Menor cobertura "enterprise"; dependiente del móvil.	Menor foco en gestión integral de costos/combustible vs. suites completas.
	Oportunidades	Digitalización del sector y integraciones (facturación, GPS).	Creciente adopción SaaS en LATAM/SMBs.	Extender funciones de flota ligera/pequeñas empresas.	Integrarse con plataformas de gestión de flotas y TMS.
	Amenazas	UX inicial: si es compleja, podría afectar adopción temprana.	Competidores de menor costo o suites verticales.	Usuarios migran a soluciones más completas a medida que crecen.	Cambios normativos que reduzcan inspecciones obligatorias.

2.1.2. Estrategias y tácticas frente a competidores

Para competir eficazmente frente a suites de gestión de flotas, apps personales de control vehicular y plataformas de inspecciones, Autonexo aplicará las siguientes estrategias y tácticas preliminares, considerando nuestras fortalezas (gestión integral del mantenimiento y conexión conductor–mecánico) y la oportunidad de mercado (demanda creciente de transparencia, trazabilidad y control).

Diferenciación por enfoque especializado en mantenimiento

Estrategia: Posicionar a Autonexo como la plataforma especializada en mantenimiento vehicular que conecta a conductores/flotas con mecánicos locales, y que centraliza historial, presupuestos y comunicación.

Tácticas:

- Desarrollar el buscador de mecánico con filtros por especialidad, SLA y calificación.
- Ofrecer historial técnico unificado (diagnóstico, OT, evidencias) compatible entre taller y conductor.
- Implementar mensajería in-app y cotizaciones rápidas con plantillas.

Modelo de precios escalables y accesibles

Estrategia: Mantener un esquema competitivo con planes por taller/mecánico (nº de vehículos, módulos) y freemium para conductores, mejorando relación valor/precio frente a suites completas.

Tácticas:

- Prueba gratuita (n días) y descuento por anualidad; cupones por referidos.
- Add-ons (p. ej., inventario, analítica avanzada, multi-sede).
- Opción pay-per-lead para talleres en el marketplace.

Mejora progresiva del producto y adaptabilidad

Estrategia: Competir con un ritmo de entrega ágil y modular, priorizando feedback de talleres piloto.

Tácticas:

- Sprints mensuales con roadmap público y Beta Program.
- Importadores desde Excel/WhatsApp para acelerar onboarding.

Seguridad, confianza y cumplimiento

Estrategia: Reducir la percepción de riesgo y aumentar la confianza desde etapas tempranas.

Tácticas:

- Cifrado en tránsito y en reposo, backups automáticos, control de accesos por roles.
- Auditoría básica externa y pruebas de penetración anuales.
- Bitácora de cambios en historiales y descarga de datos (portabilidad) para transparencia.

Ecosistema e integraciones

Estrategia: Construir un ecosistema digital que conecte mantenimiento y operaciones para optimizar decisiones con datos.

Tácticas:

- APIs RESTful para módulos internos; conectores con facturación, pasarelas de pago, GPS/telemática y WhatsApp Business.
- Alertas inteligentes (kilometraje/horas, tiempo, códigos OBD, garantías de repuesto).
- Dashboard de salud de flota y forecasts de mantenimiento.

Alianzas estratégicas

Estrategia: Acelerar adquisición con presencia local y pruebas guiadas.

Tácticas:

- Alianzas con cadenas de talleres, aseguradoras/SOAT y escuelas de mecánica.
- Workshops "Digitaliza tu taller en 3 pasos con Autonexo" y casos de éxito locales.
- Programa de Embajadores de Taller (beneficios por referidos y feedback).

Adquisición y retención por educación y valor continuo

Estrategia: Minimizar barreras de adopción en segmentos poco digitalizados.

Tácticas:

- Onboarding guiado y centro de ayuda con micro-videos.
- Plantillas de presupuestos, OT y checklist de inspección.
- NPS trimestral y cohortes de retención con playbooks de reactivación.

2.2. Entrevistas

2.2.1. Diseño de entrevistas

Se realizó investigación cualitativa mediante entrevistas a los segmentos objetivo del proyecto: mecánicos/talleres y conductores de vehículos. El objetivo fue comprender las herramientas, procesos y problemas actuales en su gestión de mantenimiento vehicular, y validar posibles soluciones digitales basadas en sus experiencias.

Se desarrollaron dos bloques de preguntas, diferenciados por el segmento objetivo. Las preguntas buscaron recopilar tanto información objetiva (contexto de trabajo, herramientas utilizadas) como información subjetiva (percepciones, frustraciones y expectativas respecto a posibles soluciones).

Segmento 1: Propietarios de vehículos

Preguntas sobre la problemática

1. ¿Cuáles son tus principales problemas al buscar o gestionar mantenimientos de tu vehículo?
2. ¿Cómo sueles pedir actualmente un mantenimiento y en qué parte del proceso se complica más?

Preguntas sobre la solución

1. ¿Cómo te gustaría crear la solicitud de mantenimiento en la app (voz, formulario, chat, llamada)?
2. ¿Qué información mínima debería incluir tu solicitud (placa, síntomas, fotos/video, ubicación, presupuesto)?
3. ¿Cómo preferirías ver y comparar mecánicos (lista, mapa, filtros) y qué criterios decidirían tu elección?
4. ¿Qué datos o preferencias te gustaría poder personalizar (mecánico favorito, horarios, precio máximo)?
5. ¿Qué formato de interfaz te resultaría más claro para revisar el estado del trabajo (checklist, porcentaje, fotos)?
6. ¿Qué información debería guardarse en el historial de tu vehículo y cómo te gustaría consultarla?
7. ¿Cómo prefieres realizar el pago de los servicios (Yape/Plin, tarjeta, efectivo) y en qué momento del proceso?
8. ¿Qué criterios usarías para calificar al mecánico después de un servicio (puntualidad, precio, confianza, claridad)?
9. Al abrir la app, ¿qué información te gustaría ver primero (urgencias, citas del día, presupuestos pendientes)?

Segmento 2: Mecánicos

Preguntas sobre la problemática

1. ¿Cuáles son los problemas más comunes que enfrentas al gestionar los mantenimientos de los vehículos?
2. ¿Qué dificultades actuales en el mantenimiento de vehículos crees que la app debería resolver primero para facilitar tu trabajo?

Preguntas sobre la solución

1. ¿Qué procesos consideras esenciales para que una app te ayude a administrar las solicitudes de mantenimiento?
2. ¿Cómo te gustaría que la app recibiera las solicitudes de mantenimiento y qué información debería incluir cada solicitud?
3. ¿Qué funcionalidades te gustaría que tuviera la app para organizar y asignar los trabajos de mantenimiento entre los mecánicos disponibles?

4. ¿Qué opciones de personalización debería ofrecer la app para adaptarse a tus necesidades específicas como mecánico?
5. ¿Qué tipo de interfaz te resultaría más fácil y eficiente para consultar el estado de un vehículo y las reparaciones necesarias?
6. ¿Cómo debería la app manejar el seguimiento de los trabajos realizados y el historial de mantenimiento de cada vehículo?
7. ¿Qué funcionalidades consideras necesarias para que los clientes puedan interactuar con la app de manera clara y efectiva?
8. ¿Qué tipo de alertas o notificaciones serían más útiles para recordarte mantenimientos programados o trabajos pendientes?
9. ¿Qué opciones de pago te gustaría que la app integrara para que el cobro de los servicios sea rápido y seguro?
10. ¿Qué información o herramientas adicionales (p. ej., apoyo a diagnóstico) te gustaría que la app incluyera para hacer diagnósticos más rápidos y sugerir mantenimientos precisos?
11. ¿Qué tipo de retroalimentación o valoraciones te gustaría que los clientes pudieran dejar sobre tu trabajo?
12. ¿Qué información debería mostrar la app al inicio para ayudarte a priorizar los mantenimientos más urgentes?
13. ¿Cómo debería la app ayudarte a coordinar trabajos entre varios mecánicos si un vehículo requiere más de un tipo de mantenimiento?

2.2.2. Registro de entrevistas

Segmento 1: Propietarios de vehículos

Entrevista	1	Nombre	Diego Ignacio Ricra Falla
Edad	20	Distrito	La molina
Captura de la entrevista:  Captura de la entrevista	Diego Ricra, estudiante de 19 años, utiliza un CR-V 2012 y enfrenta problemas para encontrar talleres cercanos, confiables y con precios justos. Actualmente depende de un mecánico de confianza recomendado por terceros, pero valora la posibilidad de una app que le permita crear solicitudes personalizadas con fotos, ubicación y presupuesto. Prefiere comparar mecánicos por precio y cercanía, pagar con Yape/Plin o efectivo, y recibir actualizaciones constantes mediante fotos o checklist. Considera clave la confianza, la comunicación clara y las experiencias previas de otros usuarios. Espera que la app le brinde rapidez, transparencia y control en todo el proceso de mantenimiento		
URL de la grabación	Ver grabación		
Timing	00:00 -08:02		

Entrevista	2	Nombre	Freddy Fernandez Camacho
Edad	22	Distrito	Ate
Captura de la entrevista:  Captura de la entrevista	Carlos Fredy Camayo, conductor de un Nissan Sunny 2001, enfrenta problemas al encontrar mecánicos confiables, ya que los precios varían y no tiene un taller de confianza. Actualmente se guía por referencias en Facebook, pero valora que una app le permita contactar rápidamente, comparar precios y acceder a soluciones rápidas para problemas comunes. Prefiere personalizar con mecánico favorito, historial de mantenimientos y comparador de repuestos. Pagaría con Yape/QR y espera funciones como buscador de repuestos, citas y soluciones rápidas. Considera esenciales las recomendaciones, precios justos y calificación simple con estrellas y etiquetas.		
URL de la grabación	Ver grabación		
Timing	08:07 - 14:58		

Segmento 2: Mecánicos

Entrevista	3	Nombre	José Angel Castillo Miranda
-------------------	---	---------------	-----------------------------

Edad	40	Distrito	San Luis
Captura de la entrevista:  Captura de la entrevista	El entrevistado, ingeniero en mantenimiento mecánico, señaló como problemas principales la falta de planificación preventiva, la mala comunicación con los conductores y la ausencia de historial organizado de cada vehículo. Consideró esenciales procesos como registro de solicitudes, calendario de mantenimientos, alertas automáticas y reportes de avance. Prefiere interfaces visuales tipo panel con indicadores y acceso rápido al historial, así como notificaciones en tiempo real. También valoró funcionalidades de seguimiento de solicitudes, métodos de pago variados (digitales o tarjeta), retroalimentación clara mediante calificaciones y la posibilidad de organizar trabajos según especialidad y disponibilidad.		
URL de la grabación	Ver grabación		
Timing	14:59 - 30:08		
Entrevista	4	Nombre	Rando Calero
Edad	45	Distrito	Suquillo
Captura de la entrevista:  Captura de la entrevista	Rando Calero, mecánico especialista en la marca Scania y dueño de un taller con un año en el mercado, destacó que los principales problemas en la gestión del mantenimiento son la falta de planificación preventiva y la dependencia de recordatorios manuales o escáneres del vehículo. Actualmente utilizan WhatsApp y registros básicos para coordinar mantenimientos, pero considera que una app debería centralizar el historial por placa, facilitar la comunicación y agilizar el flujo de trabajo. También valoró que la aplicación sea sencilla, con lenguaje accesible, opciones de retroalimentación de clientes, alertas de mantenimientos próximos y reportes claros para dueños, conductores y talleres. Además, ve como oportunidad incluir funciones de promoción de talleres, estadísticas de satisfacción y un flujo automatizado que informe al cliente en cada etapa del servicio.		
URL de la grabación	Ver grabación		
Timing	30:09 - 40:34		

2.2.3. Análisis de entrevistas

Las entrevistas se realizaron entre el 5 y el 15 de septiembre de 2025 a un total de 4 participantes: dos conductores y dos mecánicos capacitados de Peru y Venezuela. El objetivo fue identificar patrones comunes en sus frustraciones, expectativas y criterios soluciones digitales en aplicaciones móviles.

Segmento: Propietarios de vehículos

Total entrevistados: 2

Edades: 19 y 21 años

Distritos: Ate y La Molina

Universidad: Estudiantes universitarios de la UPC

Vehículos: Honda CRV 2012 (familiar), Nissan Sunny 2001 (propio)

Fechas: Entre el 5 y 9 de septiembre

Características objetivas

- Tienen dificultades para encontrar mecánicos de confianza con precios justos: 2/2 (100%)
- Usan referencias de terceros / redes sociales (Facebook, recomendaciones boca a boca) para decidir mecánico: 2/2 (100%)
- Prefieren medios de pago digitales o efectivos (Yape, Plin, QR o efectivo): 2/2 (100%)

- Desean un historial digital de mantenimientos de su vehículo: 2/2 (100%)
- Consideran importante comparar precios antes de aceptar el servicio: 2/2 (100%)
- Valorán que la app permita personalizar opciones (mecánico favorito, horarios, precio máximo, repuestos): 2/2 (100%)

Características subjetivas

- Desconfianza hacia los talleres locales por precios inflados o falta de transparencia: 2/2 (100%)
- Consideran que la ubicación del mecánico es un factor clave al decidir (cercanía): 1/2 (50%)
- Priorizan la rapidez en la comunicación con el mecánico (chat, llamada, videollamada): 2/2 (100%)
- Prefieren ver el avance del trabajo con fotos o checklist para mayor control: 2/2 (100%)
- Desean que la plataforma incluya valoraciones y comentarios de otros usuarios como guía: 2/2 (100%)
- Muestran disposición a usar una app siempre que sea intuitiva y rápida: 2/2 (100%)

Segmento: Mecánicos

Total entrevistados: 2

Edades: 40 y 45 años

Distritos: San Luis y Surquillo

Experiencia:

- José Castillo: Ingeniero en mantenimiento mecánico con experiencia en diagnósticos, mantenimientos preventivos y gestión operativa.
- Rando Calero: Mecánico especialista en Scania, dueño de taller con 1 año en el mercado.

Fechas: Entre el 13 y 15 de septiembre

Características objetivas

- Identifican la falta de planificación preventiva como el principal problema de gestión: 2/2 (100%)
- Consideran esencial el historial digital de cada vehículo para diagnósticos más rápidos: 2/2 (100%)
- Utilizan actualmente métodos básicos de comunicación (WhatsApp, llamadas): 2/2 (100%)
- Valoran procesos como registro de solicitudes, seguimiento, alertas y reportes: 2/2 (100%)
- Prefieren una interfaz clara e intuitiva, con indicadores y paneles visuales: 2/2 (100%)
- Están abiertos a que los clientes dejen valoraciones y comentarios sobre el servicio: 2/2 (100%)
- Usan o han considerado usar herramientas digitales complementarias (escáner de vehículos, páginas web, redes sociales): 2/2 (100%)

Características subjetivas

- Perciben que la mala comunicación con conductores aumenta costos y retrasa soluciones: 2/2 (100%)
- Valoran la organización de solicitudes para evitar confusiones o duplicaciones: 2/2 (100%)
- Consideran clave las notificaciones automáticas de mantenimientos: 2/2 (100%)
- Desean que la app tenga lenguaje accesible y no excesivamente técnico, para facilitar la adopción: 1/2 (50%)
- Piensan que la plataforma debe incluir flujo automatizado de trabajo (inicio, mantenimiento en proceso, finalización): 1/2 (50%)
- Ven con buenos ojos la idea de un sistema de membresías o suscripción para cobros recurrentes: 1/2 (50%)
- Reconocen que las reseñas negativas pueden dañar la reputación, pero aun así valoran su utilidad para mejorar: 1/2 (50%)

2.3. Needfinding

En el siguiente apartado, analizaremos a nuestros segmentos objetivos para identificar sus necesidades y en base a esto ofrecerles soluciones óptimas a sus problemas.

2.3.1. User Personas

Las siguientes User Persona representan a los diferentes tipos de usuarios que interactúan con nuestra aplicación. Estos perfiles nos permiten comprender sus características, motivaciones, necesidades y comportamientos, lo que facilita identificar cómo se relacionan con la solución desde su contexto personal hasta el uso activo de la misma.

- User Persona de José Quispe (mecánico) User-Persona-José-Quispe

[Ver mapa user persona hecho en UxPressia guardado en Drive](#)

- User Persona de Marina Salinas (propietaria) User-Persona-Marina-Salinas

[Ver mapa user persona hecho en UxPressia guardado en Drive](#)

2.3.2. User Task Matrix

En esta sección se presenta el User Task Matrix, que concentra las tareas que los User Persona (que representan a cada segmento) realizan para cumplir sus objetivos. No confundir tareas (tasks) con opciones o características de software, pues las tareas deben ser realizadas por los segmentos independientemente de la existencia de su solución de software.

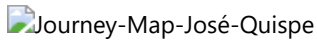
TASK	Propietario de vehículo		Mecánico	
	Frecuencia	Importancia	Frecuencia	Importancia
Iniciar sesión en la app	Always	High	Always	High
Consultar historial de mantenimiento	Often	High	Rarely	Medium
Solicitar cita de mantenimiento	Always	High	Sometimes	Medium
Confirmar cita de mantenimiento	Always	High	Always	High
Ver estado de mantenimiento	Often	High	Always	High
Dejar reseña de taller	Sometimes	Medium	Rarely	Low
Recibir notificación de avance de mantenimiento	Always	High	Always	High
Consultar disponibilidad de mecánicos	Always	High	Sometimes	Medium
Ver precios de servicios de taller	Often	High	Sometimes	Medium
Recibir confirmación de cita	Always	High	Always	High
Registrar un incidente o reclamo	Sometimes	High	Sometimes	High
Consultar mapa de talleres cercanos	Often	Medium	Rarely	Low
Ver promociones de talleres favoritos	Sometimes	Medium	Rarely	Low
Confirmar disponibilidad de citas con mecánicos	Never	Low	Always	High
Ver historial de calificaciones de talleres	Often	Medium	Rarely	Low
Ver detalles del mantenimiento realizado	Sometimes	High	Always	High
Comunicación con el propietario (mensaje/chat)	Never	Low	Always	High
Consultar disponibilidad de mecánicos por ubicación geográfica	Often	High	Sometimes	Medium
Recibir alertas de cambios en mantenimiento o citas	Sometimes	High	Always	High
Ver estado de pagos y facturación	Always	High	Never	Low

TASK	Propietario de vehículo		Mecánico	
	Frecuencia	Importancia	Frecuencia	Importancia
Actualizar datos personales (vehículo, dirección)	Rarely	Medium	Never	Low

2.3.3. User Journey Mapping

Los siguientes User Journey Maps corresponden a cada User Persona. Estos diagramas nos ayudarán a identificar y comprender las dificultades que enfrentan en cada fase, desde el momento en que conocen nuestra aplicación hasta la etapa en la que comparten sus experiencias y opiniones sobre su uso.

- User Journey Map de José Quispe (mecánico)



[Ver mapa completo hecho en UxPressia guardado en Drive](#)

- User Journey Map de Marina Salinas (propietaria)



[Ver mapa completo hecho en UxPressia guardado en Drive](#)

2.3.4. Empathy Mapping

En esta sección, se elaboró un Empathy Map para analizar a nuestros usuarios, comprendiendo completamente el entorno en el que se desenvuelven. Este proceso nos permitirá profundizar en sus necesidades y orientarnos hacia la oferta de servicios que realmente les sean útiles.

Para la construcción de los Empathy Maps, el equipo siguió los siguientes pasos:

- 1.-Preparación: Se revisó la ficha de cada User Persona creada previamente (José Quispe y María Salinas).
- 2.-Colocar al centro el User Persona: Cada mapa fue desarrollado teniendo al usuario en el centro del análisis, asegurando que sus características, contexto y metas guíen el proceso.
- 3.-Lluvia de ideas en equipo: Cada integrante compartió observaciones y percepciones sobre el comportamiento, necesidades y frustraciones de los usuarios.
- 4.-Distribución en secciones: En la herramienta de mapeo se organizaron los aportes respondiendo las preguntas clave:
 - ¿Con quién estamos empatizando?
 - ¿Qué necesita hacer?
 - ¿Qué está diciendo?
 - ¿Qué está viendo?
 - ¿Qué está haciendo?
 - ¿Qué está escuchando?
 - ¿Cómo se siente y qué piensa?
- 5.-Identificación de Pains y Gains: Se clasificaron las preocupaciones, frustraciones y motivaciones que influyen en la experiencia del usuario.
- 6.-Síntesis: Se consolidaron los hallazgos en un mapa final para cada persona.

- Empathy Map de José Quispe (mecánico)



[Ver mapa completo hecho en UxPressia guardado en Drive](#)

- Empathy Map de Marina Salinas (propietaria)

 Empathy-Map-Maria-Salinas

[Ver mapa completo hecho en UxPressia guardado en Drive](#)

2.3.5. Ubiquitous Language

1. Matching & Booking Context

- Service Request (Solicitud de servicio): Pedido inicial que realiza un cliente para atender una necesidad de mantenimiento o reparación.
- Offer (Oferta): Propuesta realizada por un taller (workshop) para cubrir un Service Request, incluyendo condiciones de precio y tiempo.
- Booking (Reserva): Confirmación de una oferta aceptada por el cliente.
- Booking Confirmation (Confirmación de reserva): Estado en el que el cliente valida definitivamente la reserva y queda registrada en el sistema.
- Booking Reschedule (Reprogramación de reserva): Cambio de la fecha o franja horaria de una reserva existente.

2. Trust & Reputation Context

- Review (Reseña): Valoración escrita y numérica de un servicio recibido por parte de un cliente.
- Trust Score (Puntaje de confianza): Métrica compuesta que refleja la reputación de un taller, un cliente o un servicio específico, en base a reseñas y comportamientos.
- Reputation (Reputación): Percepción acumulada de la calidad y confiabilidad de un taller u oferente, basada en interacciones previas.

3. Workshop Context

- Workshop (Taller): Negocio o entidad que ofrece servicios de reparación y mantenimiento de vehículos.
- Workshop Schedule (Agenda del taller): Plan de disponibilidad de un taller para aceptar reservas.
- Service Order (Orden de servicio): Documento operativo que organiza y detalla el trabajo a realizar sobre un vehículo dentro de un taller.
- Service Item (Ítem de servicio): Actividad o tarea específica incluida dentro de una orden de servicio (ej. cambio de aceite, alineación de ruedas).

4. Vehicle & Maintenance Context

- Vehicle (Vehículo): Bien físico perteneciente a un cliente, sobre el cual se realizan operaciones de mantenimiento o reparación.
- Maintenance Record (Registro de mantenimiento): Histórico auditable de todas las intervenciones realizadas a un vehículo.
- Check-in (Ingreso): Momento en el cual un vehículo es recibido en el taller para iniciar un servicio.
- Check-out (Salida): Momento en el cual un vehículo es entregado de vuelta al cliente tras completar los servicios.

5. Payment & Subscription Context

- Subscription (Suscripción): Acuerdo recurrente mediante el cual un cliente paga para acceder a beneficios o servicios de la plataforma.
- Payment (Pago): Transacción monetaria realizada por un cliente para activar o renovar una suscripción.

- Invoice (Factura): Documento que refleja el detalle de un pago realizado por un cliente.
- Billing Cycle (Ciclo de facturación): Periodo de tiempo que define cuándo corresponde realizar un cobro por la suscripción.

6. IAM (Identity & Access Management) Context

- User (Usuario): Persona que accede a la plataforma con credenciales únicas.
- Role (Rol): Conjunto de permisos asignados a un usuario que define qué acciones puede realizar.
- Permission (Permiso): Autorización concreta para ejecutar una acción en el sistema.
- Authentication (Autenticación): Proceso de verificar la identidad de un usuario mediante credenciales.
- Authorization (Autorización): Proceso de validar que un usuario tiene permisos para acceder a un recurso o realizar una acción.

7. Notifications Context

- Notification (Notificación): Mensaje enviado a uno o más destinatarios para informar sobre un evento o recordatorio.
- Channel (Canal): Medio a través del cual se entrega una notificación (Push, Email, SMS).
- Template (Plantilla): Formato predefinido de un mensaje utilizado en las notificaciones.
- Reminder (Recordatorio): Tipo de notificación programada que recuerda al cliente un evento próximo (ej. revisión del vehículo).
- Transactional Notification (Notificación transaccional): Mensaje generado automáticamente por un evento del sistema (ej. confirmación de reserva).

2.4. Requirements specification

2.4.1. User Stories

Las historias de usuario constituyen una herramienta fundamental para traducir las necesidades detectadas en entrevistas con usuarios potenciales en especificaciones funcionales del sistema. Su adecuada formulación permite estructurar requisitos claros, priorizados y orientados al usuario final, facilitando así su interpretación por parte del equipo de desarrollo. Esta práctica no solo garantiza una mejor alineación entre las expectativas del cliente y las funcionalidades desarrolladas, sino que también mejora la planificación y asignación de tareas en el ciclo de vida del software. En el caso del proyecto TrackLab, se ha definido un conjunto estructurado de épicas e historias de usuario que sirven como base para el diseño de la landing page, el desarrollo de la aplicación web y la definición de los requerimientos técnicos asociados.

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
EP01	Registro y organización inicial	Como propietario y taller, quiero registrar vehículos e historiales, para organizar la información base de mantenimiento.	No corresponde	US01, US02, US03, US04
EP02	Gestión de servicios y reservas	Como taller y propietario, quiero gestionar ofertas de mantenimiento y reservas, para asegurar que las tareas se coordinen en tiempo y forma.	No corresponde	US05, US06, US07

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
EP03	Descubrimiento y personalización	Como propietario, quiero encontrar talleres y servicios mediante filtros, ubicación y promociones, para elegir la mejor opción.	No corresponde	US08, US09, US10, US11
EP04	Comunicación y coordinación	Como usuario, quiero comunicarme y coordinar con el taller en tiempo real, para asegurar un servicio claro y sin errores.	No corresponde	US12, US13, US14
EP05	Gestión de mantenimientos	Como usuario, quiero gestionar los mantenimientos desde la creación hasta la finalización, para asegurar el control y trazabilidad de cada servicio.	No corresponde	US15, US16, US17, US18, US19
EP06	Calificaciones y reputación	Como usuarios (propietario y taller), queremos calificarnos mutuamente y consultar reputaciones, para fomentar confianza y calidad en el servicio.	No corresponde	US20, US21, US22, US23
EP07	Landing Page (visitantes)	Como visitante, quiero acceder a información, registro y soporte desde la landing page, para conocer y empezar a usar la aplicación.	No corresponde	US24, US25, US26, US27
EP08	Servicios técnicos y APIs REST	Como equipo de desarrollo, quiero exponer endpoints REST bien documentados y seguros, para facilitar la integración, el mantenimiento y la escalabilidad del sistema.	No corresponde	TS01, TS02, TS03, TS04, TS05, TS06, TS07, TS08, TS09, TS10, TS11, TS12, TS13, TS14, TS15
US01	Visualizar información y beneficios (Landing Page)	Como visitante, quiero visualizar la información general de la aplicación y sus beneficios, para entender qué ofrece antes de registrarme.	Scenario 1 — Landing informativa: • Given que un visitante accede a la landing page • When navega por la sección de información • Then el sistema (o la página) presenta claramente los beneficios, secciones y llamados a la acción con información completa sobre el servicio.	EP07

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US02	Catálogo de servicios de taller	Como taller, quiero publicar ofertas de mantenimiento (ej. cambio de aceite, frenos, afinamiento) con precio, duración y cobertura, para que los propietarios las encuentren y reserven.	Scenario 1 — Publicación válida: • Given que el taller proporciona título, descripción, precio, duración, categorías y compatibilidades mínimas • When solicita publicar la oferta • Then el sistema guarda la oferta con estado publicada, registra timestamps y la hace visible en el catálogo. Scenario 2 — Campos obligatorios incompletos: • Given que el taller omite campos obligatorios (ej. precio o categoría) • When intenta publicar la oferta • Then el sistema rechaza la publicación y devuelve la lista de campos obligatorios faltantes. Scenario 3 — Imágenes/validación: • Given que el taller adjunta imágenes que exceden límites permitidos • When intenta subir las imágenes • Then el sistema rechaza las imágenes que exceden el tamaño y acepta las válidas.	EP02

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US03	Explorar catálogo y búsqueda	Como propietario, quiero explorar y buscar ofertas de mantenimiento por palabras clave y categorías, para comparar opciones.	Scenario 1 — Búsqueda con resultados: • Given que existen ofertas que coinciden con término o categoría • When el propietario busca por palabra clave, categoría o filtro • Then el sistema devuelve una lista de ofertas coincidentes paginada y ordenable por distancia, precio, rating o relevancia. Scenario 2 — Sin resultados: • Given que no hay ofertas que coincidan con los criterios de búsqueda • When el propietario realiza la búsqueda • Then el sistema devuelve una lista vacía y sugiere alternativas (otras categorías o ampliar rango). Scenario 3 — Vista detalle: • Given que el propietario solicita info de una oferta • When solicita ver el detalle de la oferta • Then el sistema entrega la información completa de la oferta y los datos del taller (rating, ubicación, términos).	EP02
US04	Filtrar por servicio	Como usuario, quiero filtrar por categorías de servicio (ej. cambio de aceite, frenos, afinamiento), para ver solo ofertas relevantes.	Scenario 1 — Filtros básicos: • Given que existen ofertas con distintas categorías • When el usuario aplica filtros por categoría o subcategoría • Then el sistema devuelve únicamente las ofertas que cumplen los criterios seleccionados. Scenario 2 — Filtros avanzados: • Given que el usuario aplica filtros por compatibilidad (marca/motor/combustible) • When aplica dichos filtros • Then el sistema muestra las ofertas compatibles y oculta las no compatibles.	EP03

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US05	Recomendación por geolocalización	Como propietario, quiero ver talleres cercanos según rango, para coordinar fácil.	Scenario 1 — Talleres dentro del rango: • Given que existen talleres disponibles dentro del rango definido por el propietario • When solicita ver talleres cercanos • Then el sistema lista los talleres ordenados por distancia. Scenario 2 — Fallback por código postal: • Given que el propietario no permite ubicación por GPS o GPS falla • When el propietario suministra código postal o ciudad • Then el sistema busca talleres dentro del área indicada y devuelve resultados. Scenario 3 — Sin talleres en rango: • Given que no hay talleres en el rango definido • When solicita buscar • Then el sistema muestra un mensaje indicando ausencia de talleres en ese rango y sugiere ampliar búsqueda.	EP03
US06	Registro de vehículo	Como propietario, quiero registrar mi vehículo ingresando sus datos básicos, para que quede vinculado a mi perfil.	Scenario 1 — Registro exitoso: • Given que el propietario proporciona todos los datos requeridos del vehículo (marca, modelo, año, placa, etc.) • When solicita guardar el registro • Then el sistema asocia y almacena el vehículo en el perfil del propietario. Scenario 2 — Registro incompleto: • Given que el propietario omite uno o más campos obligatorios • When intenta guardar el registro • Then el sistema devuelve un error indicando los campos faltantes y no crea el registro.	EP01

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US07	Adjuntar historial inicial	Como propietario, quiero adjuntar el historial de mantenimiento de mi vehículo, para que el taller conozca trabajos previos.	Scenario 1 — Historial adjuntado: • Given que el propietario dispone de documentos o datos de mantenimientos previos • When incorpora la información o adjunta los documentos y confirma el registro del vehículo • Then el sistema guarda el historial asociado al vehículo y lo marca como disponible para talleres autorizados. Scenario 2 — Sin historial: • Given que el propietario no tiene historial disponible • When finaliza el registro del vehículo • Then el sistema crea el registro del vehículo sin historial asociado y lo deja marcado como "sin historial".	EP01
US08	Visualizar historial de vehículo	Como taller, quiero visualizar el historial de mantenimientos de un vehículo registrado, para conocer antecedentes y diagnósticos previos.	Scenario 1 — Vehículo con historial: • Given que el vehículo tiene registros previos de mantenimiento • When el taller solicita consultar el historial del vehículo • Then el sistema entrega la lista de registros previos (fechas, tipo de servicio, taller, notas) de forma completa y ordenada. Scenario 2 — Vehículo sin historial: • Given que el vehículo no tiene registros previos • When el taller solicita el historial • Then el sistema indica que no existen registros previos para ese vehículo.	EP01

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US09	Sistema de mensajería	Como usuario, quiero contar con un chat integrado, para coordinar detalles del mantenimiento.	Scenario 1 — Envío / recepción de mensajes: • Given que dos usuarios (propietario y taller) desean comunicarse • When uno envía un mensaje • Then el sistema entrega el mensaje al destinatario y registra la conversación. Scenario 2 — Adjuntar fotos: • Given que el remitente adjunta imágenes del vehículo • When envía el mensaje con adjuntos • Then el sistema acepta imágenes dentro de los límites establecidos y las asocia a la conversación. Scenario 3 — Notificación de nuevo mensaje: • Given que llega un nuevo mensaje • When el destinatario está offline o en otra sección • Then el sistema genera una notificación que informa la llegada del nuevo mensaje. Scenario 4 — Reporte de abuso: • Given que un usuario recibe mensajes inapropiados • When reporta la conversación por abuso • Then el sistema registra la denuncia y marca la conversación para revisión.	EP04

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US10	Coordinación de citas de mantenimiento	Como usuario, quiero proponer/aceptar una cita con fecha/hora, para agendar el servicio.	Scenario 1 — Proponer y aceptar cita: • Given que el taller dispone de slots y el propietario solicita servicio • When el taller propone una fecha/hora y el propietario la acepta • Then el sistema registra la cita en ambas agendas y la confirma a ambas partes. Scenario 2 — Reprogramación: • Given que existe una cita programada • When una de las partes solicita reprogramar y la otra acepta un nuevo slot disponible • Then el sistema actualiza la cita y notifica los cambios. Scenario 3 — Conflicto de agenda: • Given que el slot propuesto ya está ocupado en la agenda del taller • When se intenta confirmar una cita que choca con otra • Then el sistema rechaza la confirmación y solicita seleccionar otro slot.	EP04

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US11	Notificaciones push	Como usuario, quiero recibir notificaciones push, para estar informado de eventos relacionados con mis vehículos y mantenimientos.	Scenario 1 — Mensaje nuevo: • Given que llega un mensaje nuevo al usuario • When el mensaje es enviado por el remitente • Then el sistema notifica al destinatario mediante notificación push. Scenario 2 — Reserva confirmada: • Given que una reserva o cita es confirmada • When la confirmación queda registrada • Then el sistema notifica push a ambas partes. Scenario 3 — Avance del mantenimiento: • Given que el taller actualiza la checklist con un hito relevante • When el hito se marca como completado • Then el sistema notifica al propietario el avance del servicio.	EP04
US12	Actualización de checklist en mantenimiento	Como taller, quiero marcar tareas en la checklist de un mantenimiento en tiempo real, para registrar los avances del servicio.	Scenario 1 — Actualización de tareas (taller): • Given que el taller está realizando un mantenimiento y existe una checklist asociada • When el taller marca una o varias tareas como realizadas • Then el sistema actualiza el estado del mantenimiento y registra la marcación en la trazabilidad del servicio, además notifica el cambio al propietario.	EP05

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US13	Creación de mantenimiento confirmado	Como taller, quiero crear un mantenimiento confirmado a partir de una reserva aceptada, para dar inicio al servicio.	Scenario 1 — Creación desde reserva aceptada: • Given que el propietario ha aceptado una propuesta o reserva • When el taller confirma el inicio del servicio • Then el sistema crea el registro de mantenimiento pendiente, genera la checklist inicial y asocia la reserva al mantenimiento. Scenario 2 — Validación de datos antes de crear: • Given que el taller intenta crear un mantenimiento desde una reserva • When el sistema valida que la reserva esté en estado "aceptada" y que todos los datos requeridos estén completos • Then el sistema verifica la validez de la reserva antes de proceder con la creación del mantenimiento. Scenario 3 — Generación automática de checklist: • Given que se está creando un mantenimiento confirmado • When el sistema genera la checklist inicial basada en el tipo de servicio solicitado • Then el sistema crea automáticamente las tareas estándar para ese tipo de mantenimiento y las marca como pendientes.	EP05

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US14	Visualización de mantenimientos pendientes	Como usuario, quiero visualizar los mantenimientos pendientes, para dar seguimiento al servicio.	Scenario 1 — Taller: lista de pendientes: • Given que el taller tiene mantenimientos asignados • When consulta su listado de trabajos • Then el sistema muestra la lista de servicios pendientes con su estado y prioridades. Scenario 2 — Propietario: detalle de su mantenimiento: • Given que el vehículo del propietario está en mantenimiento • When el propietario consulta el detalle del servicio en curso • Then el sistema muestra el estado actual y la checklist asociada en tiempo real.	EP05

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US15	Visualización de checklist en mantenimiento	Como propietario, quiero visualizar en tiempo real el progreso de la checklist del mantenimiento, para conocer el avance del servicio.	Scenario 1 — Visualización Completa de Checklist: • Given que el vehículo está en mantenimiento y la checklist está completa • When el propietario solicita el detalle del servicio • Then el sistema muestra todas las tareas completadas y las pendientes, con marcas temporales de cada avance, y resalta las tareas que han sido completadas. Scenario 2 — Checklist en Progreso: • Given que el vehículo está en mantenimiento y la checklist tiene tareas en progreso • When el propietario solicita el detalle del servicio • Then el sistema muestra las tareas pendientes y las que están en progreso, con un indicador visual de progreso (por ejemplo, barra de progreso o porcentaje completado). Scenario 3 — Actualización en Tiempo Real: • Given que el vehículo está siendo atendido y las tareas de la checklist están siendo actualizadas • When el propietario visualiza la checklist • Then el sistema actualiza la información en tiempo real, reflejando cualquier cambio en el estado de las tareas sin necesidad de recargar la página.	EP05

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US16	Finalización de mantenimiento	Como taller, quiero finalizar un mantenimiento y registrar los resultados, para cerrar correctamente el servicio.	Scenario 1 — Confirmación de Finalización: • Given que el taller ha completado todas las tareas de la checklist • When el taller marca el mantenimiento como finalizado • Then el sistema solicita una confirmación de la finalización antes de cerrar el servicio, asegurando que no falten tareas. Scenario 2 — Generación de Informe de Mantenimiento: • Given que el mantenimiento ha sido finalizado y los resultados registrados • When el taller finaliza el servicio • Then el sistema genera un informe final que incluye todos los detalles del mantenimiento (trabajos realizados, repuestos, observaciones, fechas) y lo guarda para su futura consulta en el historial del vehículo.	EP05
US17	Taller favorito	Como propietario, quiero marcar un taller como favorito, para priorizarlo en futuras reservas.	Scenario 1 — Agregar a favoritos: • Given que el propietario identifica un taller que desea priorizar • When añade el taller a su lista de favoritos • Then el sistema almacena el taller en la lista de favoritos del propietario. Scenario 2 — Quitar de favoritos: • Given que el taller está en la lista de favoritos • When el propietario lo elimina • Then el sistema lo retira de la lista de favoritos.	EP03

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US18	Promociones de taller de confianza	Como propietario, quiero ver promociones de mis talleres favoritos, para decidir antes que el resto.	Scenario 1 — Promos disponibles de favoritos: • Given que existen promociones activas de los talleres favoritos del propietario • When el propietario consulta el feed de promociones • Then el sistema destaca las promociones de favoritos con datos de vigencia, cupos y precio promocional. Scenario 2 — Mostrar condiciones de promo: • Given que una promoción tiene condiciones (cupos limitados, vigencia) • When el propietario revisa la promo • Then el sistema muestra las condiciones completas y la disponibilidad actual.	EP03
US19	Cancelar reserva de mantenimiento	Como propietario, quiero cancelar una reserva de mantenimiento programada, para reprogramar si tengo un imprevisto.	Scenario 1 — Cancelación dentro de ventana permitida: • Given que la cancelación se realiza dentro de la ventana configurable (ej. >3h antes de la cita) • When el propietario solicita cancelar la reserva • Then el sistema cambia el estado a cancelada, notifica al taller y aplica la política sin penalidad. Scenario 2 — Cancelación fuera de ventana: • Given que la cancelación se realiza fuera de la ventana permitida (ej. <3h antes) • When el propietario solicita cancelar • Then el sistema rechaza o aplica la política de penalidad según reglas y notifica al taller.	EP02

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US20	Actualización automática del historial	Como taller, quiero que el historial del vehículo se actualice automáticamente al finalizar un mantenimiento, para mantener la información al día sin tener que hacerlo manualmente.	Scenario 1 — Mantenimiento finalizado: • Given que el taller marca un mantenimiento como finalizado y registra los resultados (servicios realizados, repuestos, observaciones) • When el registro de cierre queda confirmado • Then el sistema añade automáticamente ese mantenimiento al historial del vehículo y lo hace visible en el historial. Scenario 2 — Mantenimiento cancelado: • When que un mantenimiento es cancelado antes de su finalización • When el taller registra la cancelación con motivo • Then el sistema no añade el mantenimiento al historial y marca el evento como cancelado (sin entrada en historial de servicios realizados).	EP01
US21	Registro de usuario (propietario o taller)	Como visitante, quiero registrarme como propietario o taller desde la landing page, para poder empezar a usar la aplicación.	Scenario 1 — Registro exitoso: • Given que el visitante proporciona los datos necesarios y usa un email único • When confirma el registro • Then el sistema crea la cuenta, la asocia al rol elegido y envía confirmación por correo. Scenario 2 — Email duplicado: • Given que el email ya está registrado en el sistema • When el visitante intenta registrarse con ese email • Then el sistema impide la duplicación y sugiere recuperar la cuenta o usar otro email.	EP07

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US22	Preguntas frecuentes y soporte	Como visitante, quiero consultar una sección de preguntas frecuentes y soporte, para resolver dudas comunes antes de usar la aplicación.	Scenario 1 — Acceso a FAQ y contacto: • Given que el visitante necesita información o tiene una duda común • When accede a la sección de FAQ o al enlace de soporte • Then el sistema presenta respuestas categorizadas y ofrece un medio de contacto para soporte en caso de requerir atención personalizada.	EP07
US23	Contacto y descarga de la app	Como visitante, quiero acceder a una sección de contacto y call to action para descargar la app, para comunicarme con el equipo o instalar la aplicación fácilmente.	Scenario 1 — Contacto y CTA de descarga: • Given que el visitante busca contactar al equipo o descargar la app • When consulta la sección de contacto y descarga • Then el sistema (la landing) muestra un formulario de contacto funcional y enlaces claros para descargar la app en tiendas compatibles. Scenario 2 — Envío de formulario de contacto: • Given que el visitante completa el formulario de contacto con datos válidos • When envía el formulario • Then el sistema procesa el mensaje, envía confirmación al visitante y notifica al equipo de soporte sobre la nueva consulta. Scenario 3 — Descarga según dispositivo: • Given que el visitante accede desde diferentes dispositivos (iOS, Android, desktop) • When hace clic en el enlace de descarga • Then el sistema redirige automáticamente a la tienda de aplicaciones correspondiente (App Store, Google Play) o muestra opciones para ambos sistemas.	EP07

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US24	Calificación de taller	Como propietario, quiero calificar al taller después de un mantenimiento, para reflejar la calidad del servicio recibido.	Scenario 1 — Calificación válida: • Given que el mantenimiento está finalizado y el servicio está asociado al propietario • When el propietario registra una calificación (1–5) y un comentario para ese servicio • Then el sistema guarda la reseña, la asocia al servicio y actualiza la calificación promedio del taller. Scenario 2 — Restricción de una reseña por servicio: • Given que el propietario ya calificó ese servicio • When intenta agregar una segunda reseña para el mismo servicio • Then el sistema impide la duplicación y sugiere editar la reseña existente.	EP06

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US25	Calificación de propietario	Como taller, quiero calificar al propietario después de un mantenimiento, para reflejar mi experiencia trabajando con él.	Scenario 1 — Calificación de propietario: • Given que el mantenimiento fue finalizado y corresponde al taller que presta el servicio • When el taller registra una calificación y comentario sobre el propietario para ese servicio • Then el sistema guarda la evaluación y la asocia al perfil del propietario, contribuyendo a su reputación. Scenario 2 — Restricción de calificación duplicada: • Given que el taller ya calificó al propietario para un servicio específico • When intenta calificar nuevamente al mismo propietario para el mismo servicio • Then el sistema impide la calificación duplicada y sugiere editar la calificación existente. Scenario 3 — Calificación con comentario opcional: • Given que el taller desea calificar al propietario • When proporciona una calificación numérica (1-5) y opcionalmente un comentario • Then el sistema acepta la calificación con o sin comentario y actualiza la reputación promedio del propietario.	EP06

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US26	Consulta de reputación de taller	Como propietario, quiero consultar la reputación de un taller antes de reservar, para decidir si es confiable.	Scenario 1 — Consulta de reputación: • Given que el taller tiene reseñas y calificaciones previas • When el propietario solicita la reputación del taller • Then el sistema muestra calificación promedio, número de reseñas, comentarios recientes y total de servicios realizados, con opciones de ordenar/comparar. Scenario 2 — Filtro por Rango de Calificación: • Given que el propietario desea consultar talleres con cierta calificación • When el propietario aplica un filtro de calificación • Then el sistema muestra únicamente los talleres que cumplen con el rango de calificación seleccionado (por ejemplo, calificación de 4 estrellas o más).	EP06

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
US27	Consulta de reputación de propietario	Como taller, quiero consultar la reputación de un propietario antes de aceptar su reserva, para saber si es confiable.	Scenario 1 — Consulta de reputación del propietario: • Given que el propietario tiene calificaciones previas de otros talleres • When el taller solicita la reputación del propietario asociada a la reserva • Then el sistema muestra la calificación promedio, comentarios y número de mantenimientos previos. Scenario 2 — Propietario sin historial: • Given que el propietario no tiene calificaciones previas en el sistema • When el taller consulta la reputación del propietario • Then el sistema indica que es un usuario nuevo sin historial de calificaciones y muestra información básica del perfil. Scenario 3 — Filtrado por tipo de servicio: • Given que el propietario tiene calificaciones de diferentes tipos de servicios • When el taller consulta la reputación específica para el tipo de servicio solicitado • Then el sistema muestra calificaciones relevantes al tipo de servicio y un promedio específico para esa categoría.	EP06

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS01	Implementar endpoint para registrar y gestionar vehículos (POST, PUT, DELETE /vehicles)	Como developer, quiero exponer endpoints para registrar, actualizar y eliminar vehículos, para que los propietarios puedan gestionar sus datos básicos.	Scenario 1 — Registro exitoso: • Given que el cuerpo contiene datos válidos • When se envía la solicitud • Then se responde con 201 Created y el vehículo queda vinculado al propietario. Scenario 2 — Registro incompleto: • Given que faltan campos obligatorios • When se envía la solicitud • Then se responde con 400 Bad Request indicando los campos faltantes. Scenario 3 — Actualización exitosa: • Given que el ID del vehículo existe • When se actualiza con datos válidos • Then se responde con 200 OK y los datos se reflejan. Scenario 4 — Eliminación exitosa: • Given que el ID existe • When se solicita eliminar • Then se responde con 204 No Content. Scenario 5 — ID no encontrado: • Given que el vehículo no existe • When se intenta actualizar o eliminar • Then se responde con 404 Not Found.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS02	Implementar endpoint para adjuntar y consultar historial de mantenimiento (POST, GET /vehicles/{id}/history)	Como developer, quiero exponer endpoints para adjuntar y consultar historiales, para que talleres y propietarios accedan a información previa de mantenimientos.	Scenario 1 — Historial adjuntado exitosamente: • Given que el cuerpo tiene documentos válidos • When se envía la solicitud • Then se responde con 201 Created y se vincula al vehículo. Scenario 2 — Consulta de historial existente: • Given que el vehículo tiene historial • When se consulta el endpoint • Then se responde con 200 OK y la lista de mantenimientos. Scenario 3 — Vehículo sin historial: • Given que no hay registros previos • When se consulta el historial • Then se responde con 204 No Content. Scenario 4 — Vehículo no encontrado: • Given que el ID no existe • When se consulta el historial • Then se responde con 404 Not Found.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS03	Implementar endpoint para catálogo de servicios de taller (POST, PUT, DELETE, GET /services)	Como developer, quiero exponer endpoints para publicar, actualizar, eliminar y listar servicios de los talleres, para que los propietarios puedan explorarlos.	Scenario 1 — Publicación válida: • Given que los datos requeridos son completos • When se publica el servicio • Then se responde con 201 Created. Scenario 2 — Campos incompletos: • Given que faltan datos obligatorios • When se envía la solicitud • Then se responde con 400 Bad Request. Scenario 3 — Actualización exitosa: • Given que el servicio existe • When se actualiza con datos válidos • Then se responde con 200 OK. Scenario 4 — Eliminación exitosa: • Given que el servicio existe • When se solicita eliminar • Then se responde con 204 No Content. Scenario 5 — ID no encontrado: • Given que el servicio no existe • When se intenta actualizar o eliminar • Then se responde con 404 Not Found.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS04	Implementar endpoint para gestión de reservas de mantenimiento (POST, PUT, DELETE, GET /reservations)	Como developer, quiero exponer endpoints para crear, actualizar, cancelar y consultar reservas, para que talleres y propietarios puedan coordinar mantenimientos.	Scenario 1 — Reserva creada exitosamente: • Given que los datos son válidos • When se crea una reserva • Then se responde con 201 Created. Scenario 2 — Cancelación dentro de ventana permitida: • Given que la cancelación cumple las reglas • When se solicita cancelar la reserva • Then se responde con 200 OK y estado "cancelada". Scenario 3 — Cancelación fuera de ventana: • Given que la cancelación no cumple la política • When se solicita cancelar • Then se responde con 409 Conflict o la política definida. Scenario 4 — Reserva no encontrada: • Given que el ID no existe • When se consulta o cancela la reserva • Then se responde con 404 Not Found.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS05	Implementar endpoint para sistema de mensajería (POST, GET /messages)	Como developer, quiero exponer endpoints para enviar y consultar mensajes, para que propietarios y talleres puedan coordinarse vía chat.	Scenario 1 — Envío exitoso: • Given que el cuerpo es válido • When se envía un mensaje • Then se responde con 201 Created y se almacena la conversación. Scenario 2 — Adjuntar imágenes: • Given que se adjuntan imágenes dentro de límites permitidos • When se envía el mensaje • Then se responde con 201 Created y archivos asociados. Scenario 3 — Consulta de mensajes: • Given que existen mensajes previos • When se consulta la conversación • Then se responde con 200 OK y la lista. Scenario 4 — Conversación inexistente: • Given que no hay mensajes • When se consulta la conversación • Then se responde con 204 No Content.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS06	Implementar endpoint para calificaciones y reputación (POST, GET /ratings)	Como developer, quiero exponer endpoints para registrar calificaciones y consultar reputaciones de talleres/propietarios, para fomentar confianza en el sistema.	Scenario 1 — Calificación registrada exitosamente: • Given que los datos son válidos • When se registra la reseña • Then se responde con 201 Created. Scenario 2 — Calificación duplicada: • Given que ya existe una reseña para el mismo servicio • When se intenta registrar otra • Then se responde con 409 Conflict. Scenario 3 — Consulta de reputación existente: • Given que el taller o propietario tiene reseñas • When se consulta la reputación • Then se responde con 200 OK con estadísticas. Scenario 4 — Sin calificaciones previas: • Given que no existen reseñas • When se consulta la reputación • Then se responde con 204 No Content.	EP08
TS07	Implementar endpoint para exploración y búsqueda de servicios (GET /services?filters=)	Como developer, quiero exponer un endpoint que permita buscar y filtrar servicios por palabras clave, categorías, compatibilidad y orden, para que los propietarios encuentren opciones fácilmente.	Scenario 1 — Búsqueda con resultados: • Given que existen servicios que coinciden con los filtros • When se consulta el endpoint • Then se responde con 200 OK con arreglo de servicios. Scenario 2 — Sin resultados: • Given que no existen coincidencias • When se consulta el endpoint • Then se responde con 200 OK con un arreglo vacío y sugerencias. Scenario 3 — Vista detalle de servicio: • Given que se solicita ver un servicio específico • When se consulta el detalle • Then se responde con 200 OK con la información completa.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS08	Implementar endpoint para filtros y geolocalización de talleres (GET /workshops? location=)	Como developer, quiero exponer un endpoint para filtrar talleres por servicio, ubicación (GPS/código postal) y compatibilidad, para que los propietarios tengan resultados relevantes.	Scenario 1 — Talleres dentro del rango: • Given que existen talleres en el rango definido • When se consulta el endpoint • Then se responde con 200 OK con lista ordenada por distancia. Scenario 2 — GPS no disponible: • Given que no se puede usar la ubicación por GPS • When se consulta con código postal • Then se responde con 200 OK con talleres en esa zona. Scenario 3 — Sin talleres: • Given que no hay talleres disponibles • When se realiza la búsqueda • Then se responde con 200 OK con un arreglo vacío y sugerencia de ampliar rango.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS09	Implementar endpoint para gestión de favoritos (POST, DELETE, GET /favoritos)	Como developer, quiero exponer endpoints para agregar, eliminar y listar talleres favoritos, para que los propietarios puedan priorizarlos.	Scenario 1 — Agregar taller a favoritos: • Given que el taller existe • When se envía la solicitud de agregar • Then se responde con 201 Created. Scenario 2 — Quitar de favoritos: • Given que el taller está en la lista de favoritos • When se envía la solicitud de eliminar • Then se responde con 204 No Content. Scenario 3 — Listar favoritos: • Given que el usuario tiene talleres favoritos • When se consulta el endpoint • Then se responde con 200 OK con la lista. Scenario 4 — Taller no encontrado: • Given que el taller no existe • When se consulta o elimina • Then se responde con 404 Not Found.	EP08
TS10	Implementar endpoint para promociones de talleres (GET /promotions)	Como developer, quiero exponer un endpoint para listar promociones de talleres, con especial énfasis en los favoritos del propietario.	Scenario 1 — Promociones disponibles: • Given que existen promociones activas • When se consulta el endpoint • Then se responde con 200 OK con condiciones. Scenario 2 — Sin promociones activas: • Given que no existen promociones • When se consulta el endpoint • Then se responde con 204 No Content.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS11	Implementar endpoint para coordinación de citas (POST, PUT /appointments)	Como developer, quiero exponer endpoints para proponer, aceptar y reprogramar citas de mantenimiento, para sincronizar agendas.	Scenario 1 — Cita propuesta y aceptada: • Given que el taller propone una cita y el propietario la acepta • When se confirma la solicitud • Then se responde con 201 Created. Scenario 2 — Reprogramación exitosa: • Given que existe una cita programada • When se solicita reprogramar con un nuevo slot válido • Then se responde con 200 OK. Scenario 3 — Conflicto de agenda: • Given que el slot ya está ocupado • When se intenta confirmar • Then se responde con 409 Conflict.	EP08
TS12	Implementar endpoint para notificaciones push (POST /notifications)	Como developer, quiero exponer un endpoint que dispare notificaciones push al móvil/web, para informar sobre mensajes, reservas y mantenimientos.	Scenario 1 — Mensaje nuevo: • Given que un usuario recibe un mensaje • When el sistema lo registra • Then se envía una notificación push. Scenario 2 — Reserva confirmada: • Given que se confirma una reserva • When el sistema guarda la confirmación • Then se envía una notificación push a ambas partes. Scenario 3 — Avance checklist: • Given que el taller actualiza un hito de la checklist • When el cambio se guarda • Then se envía una notificación push al propietario.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS13	Implementar endpoint para gestión de mantenimientos (POST, PUT, GET, DELETE /maintenances)	Como developer, quiero exponer endpoints para crear, listar, actualizar (checklist, estado) y finalizar mantenimientos, para garantizar trazabilidad.	Scenario 1 — Creación desde reserva: • Given que existe una reserva aceptada • When el taller confirma el inicio • Then se responde con 201 Created. Scenario 2 — Actualización de checklist: • Given que existe un mantenimiento activo • When se marcan tareas como realizadas • Then se responde con 200 OK. Scenario 3 — Visualizar mantenimientos pendientes: • Given que existen mantenimientos activos • When se consulta la lista • Then se responde con 200 OK. Scenario 4 — Finalización: • Given que todas las tareas están completadas • When se marca como finalizado • Then se responde con 200 OK con registro completo. Scenario 5 — ID no encontrado: • Given que el mantenimiento no existe • When se consulta o actualiza • Then se responde con 404 Not Found.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS14	Implementar endpoint para landing page y registro de usuarios (POST, GET /users)	Como developer, quiero exponer endpoints para registrar propietarios/talleres desde la landing y obtener datos de usuarios, para iniciar la experiencia digital.	Scenario 1 — Registro exitoso: • Given que el visitante proporciona datos válidos • When se envía la solicitud • Then se responde con 201 Created con rol asignado. Scenario 2 — Email duplicado: • Given que el email ya está registrado • When se intenta registrar • Then se responde con 409 Conflict. Scenario 3 — Consulta de usuario existente: • Given que el usuario existe • When se consulta su información • Then se responde con 200 OK. Scenario 4 — Usuario no encontrado: • Given que el usuario no existe • When se consulta el endpoint • Then se responde con 404 Not Found.	EP08

Story ID	Título	Descripción	Acceptance Criteria	Linked ID
TS15	Implementar endpoint para soporte, FAQ y contacto (GET /faq, POST /support, POST /contact)	Como developer, quiero exponer endpoints que devuelvan FAQ, gestionen tickets de soporte y permitan contacto, para resolver dudas de visitantes.	Scenario 1 — FAQ con registros: • Given que existen entradas en la FAQ • When se consulta el endpoint • Then se responde con 200 OK con la lista. Scenario 2 — FAQ vacío: • Given que no hay registros en FAQ • When se consulta el endpoint • Then se responde con 204 No Content. Scenario 3 — Ticket de soporte creado: • Given que un visitante envía un ticket válido • When se procesa la solicitud • Then se responde con 201 Created. Scenario 4 — Contacto enviado: • Given que un visitante completa el formulario de contacto • When se envía la solicitud • Then se responde con 201 Created.	EP08

Spike 1: Investigación de la Integración de Stripe para el Procesamiento de Pagos

Story ID	User	Priority	Epic
SP-01	Developer	Alta	Autenticación

Title: Investigación de la Integración de Stripe para el Procesamiento de Pagos

Description:

Como equipo de desarrollo, queremos investigar cómo integrar Stripe para el procesamiento de pagos en la plataforma Autonexo, para asegurarnos de que sea una solución adecuada para manejar pagos de manera segura y eficiente.

Acceptance Criteria:

- **Given** acceso a la documentación de la API de Stripe, **When** se revisan las opciones disponibles para el procesamiento de pagos (como Payments, Checkout, Mobile SDKs, Stripe.js), **Then** se debe definir si Stripe es adecuado para la integración en Autonexo.
- **Given** las tarifas de Stripe, **When** se comparan los costos con otros sistemas de pagos en el mercado, **Then** se debe decidir si Stripe es la opción más económica para Autonexo.
- **Given** las opciones de suscripción de Stripe, **When** se determina si la plataforma de Autonexo requiere un modelo de suscripción mensual, **Then** se debe confirmar que Stripe es compatible con este modelo y con las necesidades de Autonexo.

- **Given** las capacidades de integración de Stripe con plataformas móviles (iOS/Android), **When** se verifica la documentación de integración con plataformas móviles a través de los Mobile SDKs de Stripe, **Then** se debe confirmar que Stripe es fácil de integrar con las aplicaciones móviles de Autonexo.
- **Given** que la integración de Stripe se prueba en un entorno de pruebas, **When** se realiza una transacción de pago utilizando un flujo de pago de prueba, **Then** se debe verificar que el pago se procese correctamente y que la información de pago se guarde de manera segura.
- **Given** que el procesamiento de pagos debe ser seguro, **When** se revisa el cumplimiento de las normativas de seguridad de Stripe (como PCI-DSS y tokenización), **Then** se debe garantizar que Stripe cumpla con los estándares de seguridad necesarios.
- **Given** que la integración de Stripe afecta tanto al frontend como al backend, **When** se verifica la compatibilidad de Stripe con el backend (Spring Boot), **Then** se debe asegurar que los endpoints RESTful y los webhooks de Stripe funcionen correctamente con el sistema backend de Autonexo.
- **Given** que la plataforma Autonexo podría tener un alto volumen de transacciones, **When** se realizan pruebas de carga en la integración de Stripe, **Then** se debe asegurar que Stripe pueda manejar un gran volumen de pagos sin afectar el rendimiento.
- **Given** que el procesamiento de pagos debe ser rápido, **When** se miden los tiempos de latencia al realizar pagos con Stripe, **Then** los tiempos de procesamiento de pagos no deben exceder los 3 segundos.
- **Given** que la integración de Stripe requiere documentación clara, **When** se documentan los pasos de integración, **Then** se debe crear un informe detallado que describa cómo integrar Stripe en Autonexo, incluyendo cualquier problema encontrado y las soluciones propuestas.
- **Given** que el flujo de pago debe ser fácil de usar para los clientes de Autonexo, **When** se prueba el flujo de pago en un entorno de pruebas con usuarios simulados, **Then** se debe asegurar que la experiencia del usuario sea intuitiva, rápida y sin errores.

Spike 2: Investigación de Google Maps para la Localización de Mecánicos

Story ID	User	Priority	Epic
SP-02	Developer	Alta	Localización

Title: Investigación de Google Maps para la Localización de Mecánicos

Description:

Como equipo de desarrollo, quiero investigar la integración de Google Maps para la localización de mecánicos, para ofrecer a los conductores la capacidad de encontrar mecánicos cercanos de manera rápida y eficiente.

Acceptance Criteria:

- **Given** que el equipo tiene acceso a la documentación de la API de Google Maps, **When** el equipo investigue las opciones para implementar mapas interactivos y localización, **Then** deberán determinar si la Google Maps API es adecuada para la integración en Autonexo.
- **Given** que el equipo ha analizado los costos asociados a Google Maps API, **When** comparen los costos con los beneficios que aportará la integración de la localización, **Then** deberán decidir si Google Maps es económicamente viable para la plataforma.
- **Given** que el equipo ha implementado la API de Google Maps para la localización, **When** realicen pruebas con la función de búsqueda de mecánicos, **Then** deberán asegurarse de que los resultados sean rápidos y precisos.
- **Given** que se ha integrado la Google Maps API, **When** realicen pruebas de rendimiento en la carga del mapa y los resultados de búsqueda, **Then** deberán garantizar que el sistema cargue los resultados en menos de 3 segundos.
- **Given** que la API de Google Maps está implementada, **When** un usuario busque mecánicos cercanos, **Then** deberán asegurarse de que la ubicación del mecánico sea precisa en el mapa.
- **Given** que Autonexo debe ser compatible con dispositivos móviles, **When** el equipo prueba la integración en dispositivos móviles (iOS/Android), **Then** deberán garantizar que la localización funcione correctamente en ambas plataformas.
- **Given** que la experiencia del usuario es crucial, **When** implementen la funcionalidad de localización en las aplicaciones móviles y web, **Then** deberán asegurarse de que la interfaz de usuario sea fácil de usar y eficiente.

- **Given** que la integración de Google Maps debe trabajar con el backend, **When** el equipo verifique la comunicación entre la API de Google Maps y el backend de Autonexo, **Then** deberán asegurarse de que los datos de ubicación se sincronicen correctamente en tiempo real.

Spike 3: Evaluación de Sistemas de Notificaciones Push

Story ID	User	Priority	Epic
SP-03	Developer	Alta	Notificaciones

Title: Evaluación de Sistemas de Notificaciones Push

Description:

Como equipo de desarrollo, quiero investigar los sistemas de notificaciones push para enviar alertas de mantenimientos próximos a los usuarios, para mejorar la experiencia del usuario y asegurarnos de que nunca se olviden de los mantenimientos.

Acceptance Criteria:

- **Given** que el equipo ha identificado varias opciones de sistemas de notificaciones push como Firebase y OneSignal, **When** investiguen las características y capacidades de cada uno, **Then** deberán decidir cuál es el más adecuado para la plataforma Autonexo.
- **Given** que el equipo ha probado la implementación de Firebase Cloud Messaging, **When** envíen una notificación push de prueba, **Then** deberán confirmar que la notificación se recibe correctamente en dispositivos iOS y Android.
- **Given** que el equipo ha implementado Firebase Cloud Messaging, **When** evalúen la facilidad de integración en el frontend y backend, **Then** deberán determinar si la integración es sencilla y eficiente para la plataforma.
- **Given** que el equipo ha implementado la función de notificaciones push, **When** un usuario reciba una notificación sobre un mantenimiento próximo, **Then** deberán asegurarse de que la notificación sea clara, visible y llegue sin demoras.
- **Given** que el equipo ha implementado la funcionalidad de notificaciones programadas, **When** se configure una notificación para ser enviada 24 horas antes de un mantenimiento, **Then** se deberá verificar que la notificación se envíe correctamente en el momento programado.
- **Given** que se han considerado otras opciones de notificaciones push como OneSignal, **When** se investigan sus características y ventajas comparativas, **Then** deberán decidir si OneSignal es una alternativa viable a Firebase para la plataforma.
- **Given** que el equipo ha implementado la función de notificaciones push, **When** se personaliza el contenido de las notificaciones (como texto, imágenes, botones), **Then** deberán asegurar que las notificaciones sean personalizables según las necesidades del usuario.
- **Given** que la plataforma Autonexo puede crecer en número de usuarios, **When** se evalúa la capacidad de Firebase y otras opciones para manejar un gran volumen de notificaciones, **Then** deberán determinar si el sistema puede escalar eficientemente sin afectar el rendimiento.
- **Given** que la seguridad de las notificaciones es crucial, **When** se revisa la seguridad en el envío de las notificaciones (como autenticación, encriptación), **Then** deberán garantizar que el sistema de notificaciones cumpla con los estándares de seguridad necesarios.

Spike 4: Investigación de Herramientas de Reportes de Mantenimiento

Story ID	User	Priority	Epic
SP-04	Developer	Alta	Reportes de Mantenimiento

Title: Investigación de Herramientas de Reportes de Mantenimiento

Description:

Como equipo de desarrollo, quiero investigar herramientas como Power BI o Tableau para generar informes detallados sobre el mantenimiento de los vehículos, para mejorar la toma de decisiones y dar a los usuarios acceso a datos importantes.

Acceptance Criteria:

- **Given** que el equipo ha identificado herramientas como Power BI y Tableau, **When** investiguen las características y capacidades de cada herramienta, **Then** deberán seleccionar la herramienta que mejor se adapte a las necesidades de informes de mantenimiento de Autonexo.
- **Given** que el equipo ha probado Power BI con datos de prueba, **When** creen un reporte básico, **Then** deberán evaluar si la herramienta proporciona visualizaciones claras y útiles para los usuarios.
- **Given** que el equipo ha probado Tableau para la creación de reportes, **When** realicen un análisis de desempeño, **Then** deberán asegurar que Tableau sea capaz de manejar grandes volúmenes de datos de manera eficiente.
- **Given** que Power BI y Tableau tienen interfaces diferentes, **When** se comparan la facilidad de uso de ambas herramientas, **Then** deberán elegir la herramienta más fácil de usar para el equipo de desarrollo y los usuarios finales.
- **Given** que ambas herramientas tienen modelos de precios distintos, **When** se comparan los costos de uso de Power BI y Tableau, **Then** deberán decidir cuál herramienta ofrece la mejor relación calidad-precio para Autonexo.
- **Given** que los informes deben adaptarse a diferentes necesidades, **When** el equipo personaliza los informes en Power BI y Tableau, **Then** deberán garantizar que las herramientas permitan una amplia personalización de los informes, incluyendo gráficos, tablas y filtros.
- **Given** que se requiere una visualización clara de los datos de mantenimiento, **When** se comparan las capacidades de visualización de Power BI y Tableau, **Then** deberán decidir cuál herramienta ofrece las mejores opciones de visualización para facilitar la toma de decisiones.
- **Given** que los datos de mantenimiento de Autonexo están almacenados en una base de datos, **When** el equipo integra Power BI y Tableau con los datos de mantenimiento, **Then** deberán asegurar que ambas herramientas puedan acceder a los datos de manera eficiente y sin errores.
- **Given** que la plataforma Autonexo puede crecer en volumen de datos, **When** el equipo evalúa la escalabilidad de Power BI y Tableau, **Then** deberán determinar si ambas herramientas pueden manejar el crecimiento futuro de los datos sin afectar el rendimiento.

Spike 5: Investigación de Integración de API de Pagos con MercadoPago

Story ID	User	Priority	Epic
SP-05	Developer	Alta	Integración de Pagos

Title: Investigación de Integración de API de Pagos con MercadoPago

Description:

Como equipo de desarrollo, quiero investigar cómo integrar la API de MercadoPago para procesar pagos, para ofrecer una solución de pagos alternativa para los usuarios en la plataforma.

Acceptance Criteria:

- **Given** que el equipo tiene acceso a la documentación de la API de MercadoPago, **When** el equipo investiga las opciones para integrar métodos de pago como tarjetas de crédito y débito, **Then** deberá definir si MercadoPago es adecuado para la integración en Autonexo.
- **Given** que el equipo ha realizado una prueba de pago con MercadoPago, **When** el equipo verifique que el proceso de pago se complete correctamente, **Then** deberá confirmar si MercadoPago es viable como solución de pagos para la plataforma.
- **Given** que MercadoPago ofrece tarifas para diferentes tipos de transacciones, **When** el equipo compare los costos de MercadoPago con otras soluciones de pagos, **Then** deberá decidir si MercadoPago es económicamente viable para Autonexo.
- **Given** que la integración de MercadoPago afecta el backend de Autonexo, **When** el equipo evalúe la compatibilidad de MercadoPago con el backend (Spring Boot), **Then** deberá garantizar que la API de MercadoPago se pueda integrar sin problemas con los servicios RESTful del backend.
- **Given** que el equipo requiere personalización en el flujo de pagos, **When** el equipo explore las opciones de personalización de pagos de MercadoPago (páginas de pago, métodos de pago), **Then** deberá determinar si

MercadoPago ofrece suficientes opciones de personalización para satisfacer las necesidades de Autonexo.

- **Given** que MercadoPago ofrece diferentes métodos de pago, **When** el equipo realice pruebas de pago con diferentes métodos (tarjetas, débito, transferencias), **Then** deberá asegurarse de que todos los métodos de pago funcionen correctamente en la plataforma.
- **Given** que la seguridad es fundamental en el procesamiento de pagos, **When** el equipo revise las medidas de seguridad que ofrece MercadoPago (encriptación, cumplimiento con PCI-DSS), **Then** deberá confirmar que MercadoPago cumple con los estándares de seguridad necesarios.
- **Given** que MercadoPago utiliza webhooks para notificar cambios de estado en pagos, **When** el equipo implemente y pruebe los webhooks de MercadoPago, **Then** deberá asegurarse de que los webhooks funcionen correctamente y sin errores.

Spike 6: Investigación de Técnicas de Caching para Mejorar el Rendimiento

Story ID	User	Priority	Epic
SP-06	Developer	Alta	Rendimiento

Title: Investigación de Técnicas de Caching para Mejorar el Rendimiento

Description:

Como equipo de desarrollo, quiero investigar técnicas de caching como Redis o Memcached, para mejorar el rendimiento de la plataforma y reducir los tiempos de carga.

Acceptance Criteria:

- **Given** que el equipo ha considerado opciones como Redis y Memcached, **When** investiguen las características y ventajas de cada uno, **Then** deberán seleccionar la opción que mejor se adapte a las necesidades de Autonexo.
- **Given** que el equipo ha implementado una solución de caching en Redis, **When** realicen pruebas de rendimiento en una función crítica, **Then** deberán verificar si el tiempo de respuesta mejora significativamente.
- **Given** que el equipo ha probado el caching de Memcached, **When** comparen los tiempos de respuesta en diferentes pruebas de carga, **Then** deberán decidir si Memcached es adecuado para el uso en Autonexo.
- **Given** que Redis y Memcached tienen implementaciones y configuraciones distintas, **When** el equipo evalúa la facilidad de integración de cada sistema en la arquitectura de Autonexo, **Then** deberán determinar cuál es más sencillo de implementar y mantener.
- **Given** que Autonexo puede crecer en volumen de usuarios y datos, **When** se evalúa la escalabilidad de Redis y Memcached, **Then** deberán garantizar que la opción seleccionada pueda manejar un crecimiento significativo sin afectar el rendimiento.
- **Given** que la seguridad es un factor crítico en la integración de caching, **When** se revisan las medidas de seguridad de Redis y Memcached (como la encriptación de datos), **Then** deberán confirmar que la solución de caching cumple con los requisitos de seguridad necesarios.
- **Given** que las soluciones de caching deben ser eficientes en el uso de recursos, **When** se mide el consumo de CPU y memoria al implementar Redis o Memcached, **Then** deberán asegurar que la solución elegida no afecte negativamente los recursos de Autonexo.
- **Given** que la alta disponibilidad es crucial para el sistema de caching, **When** se simulan fallos en Redis o Memcached, **Then** deberán verificar que la solución seleccionada pueda recuperarse correctamente sin perder datos importantes.

Spike 7: Evaluación de Herramientas de Seguridad para la Plataforma

Story ID	User	Priority	Epic
SP-07	Security Team	Alta	Seguridad

Descripción:

Como equipo de seguridad, quiero investigar herramientas de seguridad como OWASP ZAP o Burp Suite para realizar pruebas de penetración y asegurarnos de que la plataforma esté segura.

Criterios de Aceptación:

- **Given** que el equipo ha considerado herramientas como OWASP ZAP y Burp Suite, **When** investiguen las características de cada una, **Then** deberán decidir cuál es la mejor herramienta para realizar pruebas de seguridad en Autonexo.
- **Given** que el equipo ha implementado una prueba de seguridad con OWASP ZAP, **When** realicen un escaneo de vulnerabilidades, **Then** deberán identificar todas las posibles brechas de seguridad en el sistema.
- **Given** que el equipo ha implementado Burp Suite para pruebas de seguridad, **When** realicen pruebas de penetración, **Then** deberán asegurarse de que la plataforma esté segura contra ataques comunes.
- **Given** que OWASP ZAP y Burp Suite tienen interfaces y configuraciones distintas, **When** el equipo evalúa la facilidad de uso de ambas herramientas, **Then** deberán decidir cuál herramienta es más fácil de integrar y utilizar en el flujo de trabajo de Autonexo.
- **Given** que el objetivo es detectar vulnerabilidades críticas en la plataforma, **When** el equipo prueba las capacidades de detección de vulnerabilidades de OWASP ZAP y Burp Suite, **Then** deberán asegurarse de que ambas herramientas detecten las vulnerabilidades clave en Autonexo.
- **Given** que las configuraciones de seguridad son esenciales en las pruebas de penetración, **When** el equipo evalúa las configuraciones de seguridad de OWASP ZAP y Burp Suite, **Then** deberán determinar si las herramientas permiten una configuración detallada y personalizada según las necesidades de Autonexo.
- **Given** que las pruebas de seguridad deben integrarse con el flujo de desarrollo de Autonexo, **When** el equipo evalúa cómo OWASP ZAP y Burp Suite se integran con las herramientas y procesos de desarrollo existentes, **Then** deberán garantizar que las herramientas se integren sin interrumpir el flujo de trabajo de desarrollo.
- **Given** que los informes de seguridad son cruciales para la corrección de vulnerabilidades, **When** el equipo revisa los informes generados por OWASP ZAP y Burp Suite, **Then** deberán asegurarse de que los informes sean claros, detallados y proporcionen las recomendaciones necesarias para corregir las vulnerabilidades encontradas.

Spike 8: Investigación de Plataforma para Gestión de Suscripciones

Story ID	User	Priority	Epic
SP-08	Developer	Alta	Suscripciones

Descripción:

Como equipo de desarrollo, quiero investigar plataformas como Recurly o Stripe Billing para gestionar suscripciones y automatizar el proceso de pagos recurrentes en la plataforma.

Criterios de Aceptación:

- **Given** que el equipo ha considerado plataformas como Recurly y Stripe Billing, **When** investiguen las funcionalidades y capacidades de cada plataforma, **Then** deberán seleccionar la plataforma más adecuada para gestionar las suscripciones de Autonexo.
- **Given** que el equipo ha probado la implementación de Recurly con datos de prueba, **When** creen un ciclo de suscripción y realicen un pago de prueba, **Then** deberán verificar que la plataforma maneje correctamente los pagos recurrentes.
- **Given** que el equipo ha probado Stripe Billing para gestionar suscripciones, **When** realicen una prueba de suscripción con Stripe Billing, **Then** deberán asegurarse de que la plataforma maneje correctamente los pagos recurrentes.
- **Given** que ambas plataformas tienen estructuras de precios diferentes, **When** se comparen los costos de Recurly y Stripe Billing, **Then** deberán decidir cuál plataforma ofrece la mejor relación calidad-precio para Autonexo.
- **Given** que la plataforma de suscripciones debe integrarse con el backend de Autonexo, **When** el equipo evalúa la facilidad de integración de Recurly y Stripe Billing con el backend (Spring Boot), **Then** deberán garantizar que ambas plataformas se integren de manera sencilla y eficiente.
- **Given** que la seguridad es fundamental en los pagos recurrentes, **When** el equipo revise las medidas de seguridad de Recurly y Stripe Billing, **Then** deberán garantizar que ambas plataformas cumplan con los estándares de seguridad necesarios, como PCI-DSS.

- **Given** que es necesario gestionar los datos de clientes y suscripciones, **When** se utilicen las herramientas de reportes de Recurly y Stripe Billing, **Then** deberán asegurarse de que las plataformas generen informes detallados y fáciles de usar sobre las suscripciones.

Spike 9: Investigación de API de Localización para Mejorar la Búsqueda de Mecánicos

Story ID	User	Priority	Epic
SP-09	Developer	Alta	Localización

Descripción:

Como equipo de desarrollo, quiero investigar APIs de localización como Google Maps, Mapbox o Here para mejorar la búsqueda de mecánicos cercanos, ofreciendo una experiencia más eficiente y precisa a los conductores.

Criterios de Aceptación:

- **Given** que el equipo ha considerado APIs como Google Maps, Mapbox o Here, **When** investiguen las características y opciones de integración de cada API, **Then** deberán seleccionar la mejor API para la localización en Autonexo.
- **Given** que el equipo ha implementado la API de Google Maps para la localización, **When** realicen pruebas de búsqueda de mecánicos cercanos, **Then** deberán asegurarse de que la función de localización sea precisa y eficiente.
- **Given** que el equipo ha implementado la API de Mapbox para la localización, **When** realicen pruebas de búsqueda de mecánicos cercanos, **Then** deberán asegurarse de que la función de localización sea precisa y eficiente.
- **Given** que el equipo ha implementado la API de Here para la localización, **When** realicen pruebas de búsqueda de mecánicos cercanos, **Then** deberán asegurarse de que la función de localización sea precisa y eficiente.
- **Given** que cada API tiene diferentes modelos de precios, **When** comparen los costos de Google Maps, Mapbox y Here, **Then** deberán decidir cuál API ofrece la mejor relación calidad-precio para la plataforma.
- **Given** que Google Maps, Mapbox y Here tienen implementaciones y configuraciones distintas, **When** el equipo evalúe la facilidad de integración de cada API con la plataforma Autonexo, **Then** deberán determinar cuál API es más sencilla de implementar y mantener.
- **Given** que la plataforma Autonexo puede crecer en volumen de usuarios y mecánicos, **When** el equipo evalúe la escalabilidad de Google Maps, Mapbox y Here, **Then** deberán asegurarse de que todas las APIs puedan manejar un gran volumen de solicitudes sin afectar el rendimiento.
- **Given** que la precisión es clave en la búsqueda de mecánicos cercanos, **When** el equipo pruebe las APIs de Google Maps, Mapbox y Here en diferentes ubicaciones, **Then** deberán asegurarse de que todas las APIs proporcionen ubicaciones precisas y fiables.
- **Given** que las aplicaciones de Autonexo están disponibles en dispositivos móviles, **When** el equipo evalúe la compatibilidad de Google Maps, Mapbox y Here con dispositivos móviles (iOS/Android), **Then** deberán asegurarse de que todas las APIs funcionen correctamente en plataformas móviles.
- **Given** que tanto Google Maps, Mapbox y Here ofrecen características adicionales (como rutas optimizadas, tráfico en tiempo real, etc.), **When** el equipo explore estas características, **Then** deberán decidir si alguna de estas funcionalidades adicionales es relevante y útil para la plataforma Autonexo.

2.4.2. Impact Mapping

 Journey-Map-José-Quispe

[Ver mapa completo hecho en UxPressia guardado en Drive](#)

2.4.3. Product Backlog

Orden	User Story ID	Título	Story Points
-------	---------------	--------	--------------

Orden	User Story ID	Título	Story Points
1	US24	Visualizar información y beneficios (Landing Page)	2
2	US05	Catálogo de servicios de taller	5
3	US06	Explorar catálogo y búsqueda	5
4	US08	Filtrar por servicio	3
5	US09	Recomendación por geolocalización	5
6	US01	Registro de vehículo	3
7	US02	Adjuntar historial inicial	2
8	US03	Visualizar historial de vehículo	3
9	US12	Sistema de mensajería	5
10	US13	Coordinación de citas de mantenimiento	5
11	US14	Notificaciones push	3
12	US15	Actualización de checklist en mantenimiento	3
13	US16	Creación de mantenimiento confirmado	3
14	US17	Visualización de mantenimientos pendientes	3
15	US18	Visualización de checklist en mantenimiento	2
16	US19	Finalización de mantenimiento	3
17	US10	Taller favorito	2
18	US11	Promociones de taller de confianza	3
19	US07	Cancelar reserva de mantenimiento	3
20	US04	Actualización automática del historial	3
21	US25	Registro de usuario (propietario o taller)	3
22	US26	Preguntas frecuentes y soporte	2
23	US27	Contacto y descarga de la app	2
24	US20	Calificación de taller	2
25	US21	Calificación de propietario	2
26	US22	Consulta de reputación de taller	2
27	US23	Consulta de reputación de propietario	2
28	TS01	Implementar endpoint para registrar y gestionar vehículos (/vehicles)	5
29	TS02	Implementar endpoint para adjuntar y consultar historial (/vehicles/{id}/history)	5
30	TS03	Implementar endpoint para catálogo de servicios de taller (/services)	5
31	TS04	Implementar endpoint para gestión de reservas de mantenimiento (/reservations)	5
32	TS05	Implementar endpoint para sistema de mensajería (/messages)	5
33	TS06	Implementar endpoint para calificaciones y reputación (/ratings)	3

Orden	User Story ID	Título	Story Points
34	TS07	Implementar endpoint para exploración y búsqueda de servicios (/services? filters=)	5
35	TS08	Implementar endpoint para filtros y geolocalización de talleres (/workshops? location=)	5
36	TS09	Implementar endpoint para gestión de favoritos (/favorites)	3
37	TS10	Implementar endpoint para promociones de talleres (/promotions)	2
38	TS11	Implementar endpoint para coordinación de citas (/appointments)	5
39	TS12	Implementar endpoint para notificaciones push (/notifications)	3
40	TS13	Implementar endpoint para gestión de mantenimientos (/maintenances)	5
41	TS14	Implementar endpoint para landing page y registro de usuarios (/users)	3
42	TS15	Implementar endpoint para soporte, FAQ y contacto (/faq, /support, /contact)	2

2.5. Strategic-Level Domain-Driven Design

2.5.1. EventStorming

El objetivo de la sesión de EventStorming fue realizar una exploración amplia y colaborativa del dominio del problema de Autonexo. Mediante esta técnica buscamos representar de manera visual y comprensible los distintos eventos que ocurren dentro del flujo de interacción entre usuarios (propietarios de vehículos) y mecánicos, con el fin de identificar oportunidades de mejora, puntos críticos y posibles cambios de estado en el sistema.

- Step 1 – Unstructured Exploration: El equipo colocó en Miro todos los eventos relevantes sin ningún orden, con el fin de vaciar conocimiento y asegurar que ninguna parte del dominio quedara fuera.

- Step 2 – Timelines: Los eventos fueron organizados en una línea de tiempo, lo que permitió identificar la secuencia lógica de las acciones

step2_Timelines

- Step 3 – Pain Points: Se marcaron las dificultades que enfrentan los usuarios, como la demora en la asignación de mecánicos o la falta de transparencia en los precios.

step3_PainPoints

- Step 4 – Pivotal Points: Finalmente, se destacaron los eventos clave que generan cambios de estado.

step4_PivotalPoints

Esta dinámica permitió al equipo obtener una visión compartida del proceso, detectar problemas reales y reconocer eventos centrales que posteriormente servirían para delimitar los bounded contexts.

2.5.1.1 Candidate Context Discovery

El objetivo de la sesión de Candidate Context Discovery fue identificar y delimitar los bounded contexts del dominio a partir del EventStorming, con el fin de proponer una primera aproximación modular al sistema. Esto asegura que cada parte del dominio tenga responsabilidades claras, evitando la complejidad innecesaria y permitiendo un diseño más mantenible y escalable.

Se partió del resultado del EventStorming y se aplicó la técnica Look-for-pivotal-events, donde se tomaron como referencia los eventos clave mediante líneas secantes en los post-its ("Service request submitted", "Offer accepted", "Service completed", etc) para determinar los límites naturales entre contextos.



A partir de esto, se definieron los bounded contexts:

- IAM Context: Abarca la lógica de registro/autenticación de usuario
- Subscription context: Hace referencia al sistema de suscripción para acceder a las funcionalidades, mediante el cual la aplicación generará ingresos.
- Workshop Context: Engloba la lógica de los mecánicos dentro de los talleres.
- Trust&Reputation: Hace referencia al sistema de calificación a los usuarios.
- Vehicle&Maintenance: Engloba la lógica acerca de los vehículos y sus mantenimientos.
- Matching&Booking Context: Engloba el proceso de búsqueda, booking y ejecución del servicio.
- Notifications: Hace referencia a las notificaciones del sistema.

2.5.1.2 Domain Message Flows Modeling

2.5.1.3 Bounded Context Canvases

2.5.2. Context Mapping

El Context Mapping en Domain-Driven Design (DDD) representa explícitamente cómo interactúan los bounded contexts entre sí, definiendo contratos, direcciones de influencia (upstream/downstream) y patrones de integración. Este mapeo permite identificar qué contextos requieren estandarización, cuáles deben protegerse mediante traducción (Anti-Corruption Layer), y dónde preservar independencia para asegurar que cada contexto pueda evolucionar de forma predecible y segura.

En Autonexo, los contextos se organizaron considerando los siguientes patrones:

Context Map Patterns

1. Open Host Service (OHS) Un contexto upstream expone servicios mediante contratos estables (APIs, endpoints o eventos), de forma que múltiples consumidores downstream pueden integrarse sin conocer su modelo interno.

Uso en Autonexo: El IAM Context actúa como OHS, proveyendo autenticación, autorización y gestión de roles. Todos los contextos (Matching & Booking, Vehicle & Maintenance, Workshop, Subscription, Trust & Reputation, Notification) consumen estos servicios para validar identidad y permisos.



2. Customer/Supplier (C/S) Relación proveedor–cliente entre contextos. El Supplier prioriza parte de su backlog en función de las necesidades del Customer, adaptando sus capacidades para habilitar los objetivos del cliente.

Uso en Autonexo: Matching & Booking (Customer) – Workshop (Supplier): el Workshop Context provee disponibilidad de agenda y catálogo de servicios, mientras el Booking Context depende de esos datos para confirmar reservas de manera confiable.

Subscription (Customer) – Workshop (Supplier): las suscripciones otorgan beneficios como prioridad de agenda o descuentos, los cuales dependen de la configuración y condiciones publicadas por los talleres en el Workshop Context.



3. Shared Kernel (SK) Dos contexts comparten un submodelo común, usualmente entidades o identificadores críticos, que deben mantenerse consistentes para evitar duplicación y divergencia.

Uso en Autonexo: El Matching & Booking Context y el Vehicle & Maintenance Context comparten identificadores de vehículos y mantenimientos confirmados, de modo que ambos contexts operan sobre la misma referencia, garantizando consistencia y evitando duplicación de datos.

 ctxMapPatt_SharedKernel

4. Conformist (CF) El downstream adopta el modelo del upstream sin traducción, lo que facilita una integración rápida, pero sacrifica independencia, ya que hereda sus decisiones de diseño.

Uso en Autonexo: El Subscription Context se adapta directamente al modelo de Matching and Booking Context (ejemplo: estados de reserva como activa, cancelada, finalizada) sin redefinirlos ni transformarlos. Esto agiliza la integración, pero obliga a Subscription a depender de los cambios en Booking.

 ctxMapPatt_Conformist

5. Publisher/Subscriber (Event-Driven) Un contexto upstream publica eventos que otros contexts downstream consumen de manera asíncrona, desacoplando la integración. Esto permite que el publisher no dependa de los consumidores y que múltiples contexts reaccionen a un mismo evento.

Uso en Autonexo: El Notification Context actúa como subscriber de eventos generados por otros contexts: -Matching and Booking Context → publica el evento Reserva Creada. -Vehicle and Maintenance Context → publica el evento Mantenimiento Finalizado. -Trust and Reputation Context → publica el evento Calificación Registrada.

El Notification Context escucha estos eventos y envía notificaciones push a los usuarios afectados.

 ctxMapPatt_Publisher/Subscriber

2.5.3. Software Architecture

En esta sección se expone la arquitectura de software de la solución Red Carga, aplicando el enfoque del Modelo C4. Se incluyen los diagramas de Contexto, Contenedores y Despliegue, los cuales permiten comprender el sistema desde una visión general hasta su implementación en un entorno productivo. Cada nivel representa a los actores involucrados, las tecnologías clave y las interacciones con los servicios externos que forman parte del alcance del proyecto.

2.5.3.1 Software Architecture Context Level Diagrams

El siguiente diagrama muestra de manera general Autonexo conectado con los sistemas externos y los usuarios que intervienen

 Context-Diagram

2.5.3.2 Software Architecture Container Level Diagrams

El diagrama C2 profundiza en el sistema y representa la arquitectura general del software, destacando las principales tecnologías empleadas y la manera en que estas se interconectan.

 Container-Diagram

2.5.3.3 Software Architecture Deployment Diagrams

El siguiente diagrama muestra los contenedores del sistema Autonexo desplegados en el entorno de desarrollo y producción. Además, se muestran los servicios externos, la base de datos y el storage.

- Sistema en entorno de desarrollo:

 Deployment-Diagram-1

- Sistema en entorno de producción:

 Deployment-Diagram-2

2.6 Tactical-Level Domain-Driven Design

2.6.1. Bounded Context: Vehicle&Maintenance

2.6.1.1. Domain Layer

Este bounded context cubre la ejecución técnica del mantenimiento del vehículo y su registro histórico auditable, excluyendo matching/booking/pagos.

Aggregates

- **Vehicle**
Propósito: Representa un vehículo y gobierna su información técnica relevante al mantenimiento.
Atributos:
 - vehicleId: UUID
 - licensePlate: LicensePlate
 - brand: string
 - model: string
 - year: int
 - ownerId: UUID**Métodos:**
 - assignOwner(ownerId: UUID)
 - updateDetails(brand: string, model: string, year: int)**Relaciones:**
 - 1 → * con MaintenanceRecord.
- **MaintenanceRecord**
Propósito: Registro auditable del mantenimiento del vehículo; consolida solo tareas registrables en el historial.
Atributos:
 - recordId: UUID
 - vehicleId: UUID
 - openedAt: DateTime
 - closedAt: DateTime?
 - status: MaintenanceStatus**Métodos:**
 - addTask(task: MaintenanceTask)
 - approve()
 - amend(note: string)
 - complete()**Relaciones:**
 - 1 → * con MaintenanceTask.
 - 1 → 0..1 con DiagnosticReport (puede existir un informe por record).

Entities

- **MaintenanceTask**
Propósito: Tarea técnica ejecutada durante el mantenimiento. Puede ser registrable o no registrable.
Atributos:
 - taskId: UUID
 - description: string

- category: TaskCategory (enum: Maintenance, Inspection, Cleaning, Other)
- recordable: bool
- status: TaskStatus (enum: Planned, InProgress, Completed, IssueReported)

Métodos:

- markInProgress()
- markCompleted()
- reportIssue(details: string)

Relaciones:

- ■ → * con PartUsage.
- ■ → 1 con MaintenanceRecord (pertenencia).

- **PartUsage**

Propósito: Uso de repuestos/piezas dentro de una tarea.

Atributos:

- partUsageld: UUID
- taskId: UUID
- partNumber: string
- quantity: int

Métodos:

- sin reglas de negocio complejas

Relaciones:

- ■ → 1 con MaintenanceTask (pertenencia).

- **DiagnosticReport**

Propósito: Informe técnico de diagnóstico asociado al mantenimiento.

Atributos:

- reportId: UUID
- recordId: UUID
- summary: string
- createdAt: DateTime

Métodos:

- creación/generación a través de servicio de dominio DiagnosticService

Relaciones:

- 1 → 1 con MaintenanceRecord.

Value Objects

- **LicensePlate**

Propósito: Encapsula y valida la matrícula.

Atributos:

- value: string

Métodos:

- validatePlate()

Relaciones:

- Usado por Vehicle.

- **Mileage**

Propósito: Kilometraje con marca de tiempo (útil para trazabilidad del mantenimiento).

Atributos:

- value: int
- measuredAt: DateTime

Events (Domain Events)

- `VehicleCheckedInEvent(vehicleId: UUID, occurredAt: DateTime)`
Propósito: El vehículo ingresó al taller.
 - `VehicleCheckedOutEvent(vehicleId: UUID, occurredAt: DateTime)`
Propósito: El vehículo salió del taller.
 - `MaintenanceRecordOpenedEvent(recordId: UUID, vehicleId: UUID, occurredAt: DateTime)`
Propósito: Se abrió un registro de mantenimiento.
 - `MaintenanceTaskAddedEvent(recordId: UUID, taskId: UUID, recordable: bool, occurredAt: DateTime)`
Propósito: Se añadió una tarea al mantenimiento.
 - `MaintenanceTaskCompletedEvent(recordId: UUID, taskId: UUID, occurredAt: DateTime)`
Propósito: Se completó una tarea. Puede desencadenar verificación de cierre.
 - `MaintenanceTaskIssueReportedEvent(recordId: UUID, taskId: UUID, issue: string, occurredAt: DateTime)`
Propósito: Se reportó un problema en una tarea.
 - `ServiceProgressUpdatedEvent(recordId: UUID, note: string, occurredAt: DateTime)`
Propósito: Se actualizó el progreso del servicio.
 - `MaintenanceRecordApprovedEvent(recordId: UUID, occurredAt: DateTime)`
Propósito: Se aprobó el registro (cumple políticas).
 - `MaintenanceRecordAmendedEvent(recordId: UUID, note: string, occurredAt: DateTime)`
Propósito: Se anotó/ajustó el registro.
 - `MaintenanceCompletedEvent(recordId: UUID, occurredAt: DateTime)`
Propósito: Se completó el mantenimiento (tareas registrables completas).
 - `ServiceReportGeneratedEvent(recordId: UUID, reportId: UUID, occurredAt: DateTime)`
Propósito: Se generó el reporte del servicio (entregable).
-

Services (Domain Interfaces)

- `interface DiagnosticService { DiagnosticReport run(UUID recordId); }`
Propósito: Ejecuta diagnósticos técnicos y retorna un `DiagnosticReport`.
Relaciones:
 - Colabora con `MaintenanceRecord` para persistir el informe.
 - `interface MaintenancePolicy { bool canAmend(UUID recordId); bool isTaskRecordable(TaskCategory category); }`
Propósito: Reglas del dominio para determinar si es posible enmendar y cuándo una tarea es registrable.
Relaciones:
 - Consultado por `MaintenanceRecord` durante `amend(...)` y `addTask(...)`.
 - `interface PartsCatalogService { bool checkAvailability(string partNumber, int qty); }`
Propósito: Verifica disponibilidad de repuestos (sin detalles de infraestructura aquí).
Relaciones:
 - Usado por `MaintenanceTask/MaintenanceRecord` antes de confirmar `PartUsage`.
-

Commands (visión global)

Son records inmutables que expresan intención de cambio. No se documentan uno por uno en detalle por ser auxiliares; su rol global es activar métodos de agregados, validar políticas y emitir eventos.

Ejemplos:

- OpenMaintenanceRecord(recordId: UUID, vehicleId: UUID)
- AddMaintenanceTask(recordId: UUID, description: string, category: TaskCategory, recordable: bool)
- CompleteMaintenanceTask(recordId: UUID, taskId: UUID)
- ReportMaintenanceTaskIssue(recordId: UUID, taskId: UUID, issue: string)
- ApproveMaintenanceRecord(recordId: UUID) / AmendMaintenanceRecord(recordId: UUID, note: string)
- CompleteMaintenance(recordId: UUID) / GenerateServiceReport(recordId: UUID)
- CheckInVehicle(vehicleId: UUID) / CheckOutVehicle(vehicleId: UUID)

Queries (visión global)

Son records inmutables para lectura. No modifican estado; no ameritan diccionario individual.

Ejemplos:

- GetVehicleById(vehicleId: UUID)
- GetMaintenanceRecordDetails(recordId: UUID)
- ListMaintenanceRecords(vehicleId: UUID, from?: DateTime, to?: DateTime)
- GetDiagnosticReport(recordId: UUID)

2.6.1.2. Interface Layer

Capa de presentación con Controllers, Resources y Assemblers (mapeo). Dado que son auxiliares, se describe su rol global y ejemplos principales.

- **Controllers** (endpoints): VehicleController, MaintenanceRecordController.
 - Orquestan comandos/consultas y devuelven Resources.
- **Resources**
Ejemplos:
 - VehicleResource { uuid vehicleId; string licensePlate; string brand; string model; int year; uuid ownerId; }
 - OpenMaintenanceRecordResource { uuid vehicleId; }
 - MaintenanceRecordResource { uuid recordId; uuid vehicleId; string status; DateTime openedAt; DateTime? closedAt; List items; }
 - MaintenanceTaskResource { uuid taskId; string description; string category; bool recordable; string status; List parts; }
 - DiagnosticReportResource { uuid reportId; uuid recordId; string summary; DateTime createdAt; }
- **Assemblers**
 - Rol: mapear Resources $\leftrightarrow$ Commands/Entities, mantener consistencia del Ubiquitous Language.
 - Ejemplos: VehicleAssembler, MaintenanceRecordAssembler, MaintenanceTaskAssembler, DiagnosticReportAssembler.

2.6.1.3. Application Layer

Implementa servicios de aplicación y manejadores de eventos.

- **Command Services**
 - VehicleCommandService
 - MaintenanceRecordCommandService

- **Query Services**

- VehicleQueryService → GetVehicleById
- MaintenanceRecordQueryService → GetMaintenanceRecordDetails, ListMaintenanceRecords
- DiagnosticReportQueryService → GetDiagnosticReport

- **Event Handlers**

- OnMaintenanceTaskCompleted → al completar todas las tareas registrables, dispara MaintenanceCompletedEvent
- OnMaintenanceCompleted → genera ServiceReportGeneratedEvent

- **ACL / Outbound**

- CatalogAcl (consulta piezas), NotificationsAcl
- DomainEventPublisher, ReportGenerator

2.6.1.4. Infrastructure Layer

Implementaciones concretas de persistencia y adaptadores externos. Se detallan a nivel de rol (no por clase) por ser soporte técnico.

- **Repositories (implementaciones de interfaces del dominio)**

- VehicleRepository : IVehicleRepository
- MaintenanceRecordRepository : IMaintenanceRecordRepository
- DiagnosticReportRepository : IDiagnosticReportRepository
- Rol: mapeo ORM, transacciones, UoW.

- **External Services Adapters**

- PartsCatalogAdapter (catálogo de repuestos)
- EmailNotificationAdapter (notificaciones)
- MessageBrokerPublisher (publicación de eventos)

2.6.1.5 Bounded Context Software Architecture Component Level Diagrams

 Component-IAM

2.6.1.6 Bounded Context Software Architecture Code Level Diagrams

2.6.1.6.1 Bounded Context Domain Layer Class Diagrams

 Class-IAM

2.6.1.6.2 Bounded Context Database Design Diagram

 Database-IAM

2.6.2. Bounded Context: Trusting&Reputation

2.6.2.1. Domain Layer

Este bounded context cubre la gestión de reseñas, calificaciones y el cálculo de la reputación o confianza, aplicable a distintos objetivos: propietarios (owner), talleres (workshop) o service orders. Consume eventos de otros bounded contexts para validar la elegibilidad y ajustar los puntajes de confianza.

Aggregates

- **TrustProfile**

Propósito: Perfil de confianza/reputación asociado a un objetivo (owner, workshop o service order). Consolida puntajes y métricas.

Atributos:

- trustProfileId: UUID
- targetRef: TargetRef
- currentScore: ScoreValue
- ratingCount: int
- lastRecalculatedAt: DateTime
- breakdown: RatingBreakdown

Métodos:

- applyRating(rating: Rating)
- removeRating(ratingId: UUID)
- recalculate(scores: List, breakdowns: List)

Relaciones:

- 1 → * con Rating
- 1 → * con Review (opcional)

- **Review**

Propósito: Reseña textual asociada a una calificación. Puede editarse o eliminarse bajo políticas.

Atributos:

- reviewId: UUID
- targetRef: TargetRef
- authorId: UUID
- comment: Comment
- createdAt: DateTime
- editedAt: DateTime?

Métodos:

- edit(newComment: Comment)
- delete()

Relaciones:

- 1 → 1 con Rating
-

Entities

- **Rating**

Propósito: Calificación numérica dirigida a un objetivo.

Atributos:

- ratingId: UUID
- targetRef: TargetRef
- authorId: UUID
- overall: ScoreValue
- breakdown: RatingBreakdown
- createdAt: DateTime

Métodos:

- updateBreakdown(newBreakdown: RatingBreakdown)

Relaciones:

- ■ → 1 con TrustProfile
- 0..1 → 1 con Review

- **TrustAdjustment**

Propósito: Ajuste al score por eventos (fraude, cancelación, no-show).

Atributos:

- adjustmentId: UUID
- targetRef: TargetRef
- delta: decimal
- reason: string
- occurredAt: DateTime

Métodos:

- describe()

Relaciones:

- ■ → 1 con TrustProfile

Value Objects

- **TargetRef**

Propósito: Referencia polimórfica al objetivo calificado.

Atributos:

- targetType: TargetType (enum: Owner, Workshop, ServiceOrder)
- targetId: UUID

- **ScoreValue**

Propósito: Valor de puntuación acotado.

Atributos:

- value: decimal

- **RatingBreakdown**

Propósito: Desglose de puntuación por dimensiones.

Atributos:

- punctuality: ScoreValue
- quality: ScoreValue
- communication: ScoreValue
- transparency: ScoreValue

- **Comment**

Propósito: Texto validado de la reseña.

Atributos:

- text: string

Events (Domain Events)

- ReviewSubmittedEvent(reviewId, targetRef, authorId, occurredAt)
- ReviewEditedEvent(reviewId, targetRef, occurredAt)
- ReviewDeletedEvent(reviewId, targetRef, occurredAt)
- RatingSubmittedEvent(ratingId, targetRef, overall, occurredAt)
- RatingUpdatedEvent(ratingId, targetRef, occurredAt)

- TrustAdjustmentAppliedEvent(adjustmentId, targetRef, delta, occurredAt)
 - TrustScoreRecalculatedEvent(trustProfileId, targetRef, newScore, occurredAt)
 - OwnerRatedEvent(ownerId, ratingId, occurredAt)
 - WorkshopRatedEvent(workshopId, ratingId, occurredAt)
 - ServiceOrderRatedEvent(serviceOrderId, ratingId, occurredAt)
-

Services (Domain Interfaces)

- ReputationCalculator

Métodos:

- compute(ratings, adjustments) → ScoreValue
- computeBreakdown(ratings) → RatingBreakdown

- ReviewPolicy

Métodos:

- canEdit(reviewId, requesterId)
- canDelete(reviewId, requesterId)

- EligibilityPolicy

Métodos:

- canRate(targetRef, authorId)
-

Commands (visión global)

Son records inmutables que expresan intención de cambio. Activan métodos de agregados, validan políticas y emiten eventos.

Ejemplos:

- SubmitReview(targetRef, authorId, comment)
 - EditReview(reviewId, authorId, newComment)
 - DeleteReview(reviewId, authorId)
 - SubmitRating(targetRef, authorId, overall, breakdown)
 - UpdateRating(ratingId, authorId, breakdown)
 - ApplyTrustAdjustment(targetRef, delta, reason)
 - RecalculateTrustScore(targetRef)
-

Queries (visión global)

Son records inmutables de lectura. No modifican estado.

Ejemplos:

- GetTrustProfileByTarget(targetRef)
 - GetAverageScoreByTarget(targetRef)
 - ListReviewsByTarget(targetRef, from?, to?)
 - ListRatingsByTarget(targetRef, from?, to?)
-

2.6.2.2. Interface Layer

Capa de presentación con Controllers, Resources y Assemblers.

- Controllers: ReviewController, RatingController, TrustProfileController
- Resources:

- SubmitReviewResource { targetType, targetId, comment }
 - SubmitRatingResource { targetType, targetId, overall, RatingBreakdownResource }
 - TrustProfileResource { trustProfileId, targetType, targetId, currentScore, ratingCount, lastRecalculatedAt, breakdown }
 - ReviewResource { reviewId, targetType, targetId, authorId, comment, createdAt, editedAt }
 - RatingResource { ratingId, targetType, targetId, authorId, overall, breakdown, createdAt }
 - Assemblers: ReviewAssembler, RatingAssembler, TrustProfileAssembler
-

2.6.2.3. Application Layer

Implementa servicios de aplicación y manejadores de eventos.

- Command Services: ReviewCommandService, RatingCommandService, TrustProfileCommandService
 - Query Services: TrustProfileQueryService, ReviewQueryService, RatingQueryService
 - Event Handlers:
 - OnRatingSubmitted → aplica rating y recalcula TrustProfile
 - OnReviewSubmitted → vincula review con rating
 - OnTrustAdjustmentApplied → recalcula TrustProfile
 - OnServiceCompleted (externo) → habilita EligibilityPolicy
 - ACL / Outbound: BookingAcl, IdentityAcl, MaintenanceAcl, DomainEventPublisher, ReputationAnalyticsExporter
-

2.6.2.4. Infrastructure Layer

Implementaciones de persistencia y adaptadores externos.

- Repositories: TrustProfileRepository, ReviewRepository, RatingRepository, TrustAdjustmentRepository
 - External Services Adapters: MessageBrokerPublisher, IdentityProviderAdapter, AnalyticsExportAdapter
-

2.6.2.5 Bounded Context Software Architecture Component Level Diagrams

 Component-Workshops

2.6.2.6 Bounded Context Software Architecture Code Level Diagrams

2.6.2.6.1 Bounded Context Domain Layer Class Diagrams

 Class-Workshop

2.6.2.6.2 Bounded Context Database Design Diagram

 Database-Workshop

2.6.3. Bounded Context: Matching&Booking

2.6.3.1. Domain Layer

Este bounded context cubre desde la solicitud de servicio hasta la creación/confirmación del booking, la gestión de ofertas y reprogramaciones, y la apertura de la service order para el handover hacia Vehicle&Maintenance.

Aggregates

- **ServiceRequest**

Propósito: Intención del propietario de recibir un servicio (tipo, ventana deseada, ubicación).

Atributos:

- serviceRequestId: UUID
- ownerId: UUID
- vehicleId: UUID
- serviceType: ServiceType
- preferredWindow: TimeRange
- location: GeoLocation
- status: ServiceRequestStatus (Open, Matching, Offered, Archived, Cancelled)

Métodos:

- startMatching()
- archive(reason: string)
- cancel(reason: string)

Relaciones:

- 1 → * con Offer
- 1 → 0..1 con Booking

- **Offer**

Propósito: Propuesta desde un workshop (referencia externa por workshopId) para atender el ServiceRequest.

Atributos:

- offerId: UUID
- serviceRequestId: UUID
- workshopId: UUID
- proposedSlot: TimeRange
- price: Money
- terms: Terms
- status: OfferStatus (Submitted, Withdrawn, Accepted, Rejected, Expired)

Métodos:

- withdraw(reason: string)
- accept()
- reject()

Relaciones:

- ■ → 1 con ServiceRequest
- 0..1 → 1 con Booking

- **Booking**

Propósito: Reserva formal entre owner y workshop basada en una Offer aceptada.

Atributos:

- bookingId: UUID
- serviceRequestId: UUID
- offerId: UUID
- workshopId: UUID
- vehicleId: UUID
- slot: TimeRange
- status: BookingStatus (Created, Confirmed, RescheduleRequested, Rescheduled, Cancelled)
- cancellationPolicy: CancellationPolicy
- createdAt: DateTime

Métodos:

- confirm()
- requestReschedule(newSlot: TimeRange, reason: string)
- reschedule(approvedSlot: TimeRange)

- cancel(reason: string)

Relaciones:

- 1 → 1 con Offer
- 1 → 1 con ServiceRequest
- 1 → 0..1 con ServiceOrder

- **ServiceOrder**

Propósito: Orden operativa creada al confirmar un Booking; artefacto de handover hacia Vehicle&Maintenance.

Atributos:

- serviceOrderId: UUID
- bookingId: UUID
- workshopId: UUID
- vehicleId: UUID
- scheduledSlot: TimeRange
- status: ServiceOrderStatus (Opened, Closed, Cancelled)
- openedAt: DateTime

Métodos:

- open()
- close()
- cancel(reason: string)

Relaciones:

- 1 → 1 con Booking

Entities

- **RescheduleRequest**

Propósito: Solicitud de reprogramación sobre un Booking existente.

Atributos:

- rescheduleRequestId: UUID
- bookingId: UUID
- requestedSlot: TimeRange
- reason: string
- status: RescheduleStatus (Requested, Approved, Declined)

Métodos:

- approve(approvedSlot: TimeRange)
- decline(reason: string)

Relaciones:

- ■ → 1 con Booking

Value Objects

- **TimeRange:** start: DateTime, end: DateTime
- **Money:** amount: decimal, currency: string
- **GeoLocation:** lat: decimal, lon: decimal
- **Terms:** summary: string, detailsUrl: string?
- **CancellationPolicy:** freeUntilHours: int, penaltyAfter: Money
- **ServiceType:** enum (Inspection, OilChange, Tires, Brakes, Bodywork, Other)

Events (Domain Events)

- ServiceRequestSubmittedEvent(serviceRequestId, ownerId, vehicleId, serviceType, occurredAt)
- MatchingProcessStartedEvent(serviceRequestId, occurredAt)
- WorkshopRecommendationsGeneratedEvent(serviceRequestId, workshopIds, occurredAt)

- OfferSubmittedEvent(offerId, serviceRequestId, workshopId, occurredAt)
 - OfferWithdrawnEvent(offerId, occurredAt)
 - OfferAcceptedEvent(offerId, serviceRequestId, occurredAt)
 - BookingCreatedEvent(bookingId, serviceRequestId, offerId, occurredAt)
 - BookingConfirmedEvent(bookingId, occurredAt)
 - BookingRescheduleRequestedEvent(bookingId, rescheduleRequestId, requestedSlot, occurredAt)
 - BookingRescheduledEvent(bookingId, newSlot, occurredAt)
 - BookingCancelledEvent(bookingId, reason, occurredAt)
 - ServiceOrderOpenedEvent(serviceOrderId, bookingId, occurredAt)
 - ServiceOrderClosedEvent(serviceOrderId, bookingId, occurredAt)
 - ServiceRequestArchivedEvent(serviceRequestId, occurredAt)
-

Services (Domain Interfaces)

Nota: Aquí solo se definen interfaces. **Sus implementaciones que interactúan con otros BCs se ubican en Application Layer** mediante ACL/adapters.

- **MatchingEngine**

Métodos:

- findCandidates(serviceRequest: ServiceRequest) → List (workshopId)

- **AvailabilityService**

Métodos:

- isAvailable(workshopId: UUID, slot: TimeRange) → bool
- suggestSlots(workshopId: UUID, constraints: TimeRange) → List

- **PricingService**

Métodos:

- quote(serviceType: ServiceType, workshopId: UUID) → Money

- **OfferPolicy**

Métodos:

- canSubmit(workshopId: UUID, serviceRequestId: UUID) → bool
- canAccept(offerId: UUID) → bool

- **BookingPolicy**

Métodos:

- canConfirm(bookingId: UUID) → bool
- canCancel(bookingId: UUID, when: DateTime) → bool

- **ScheduleAllocator**

Métodos:

- reserve(workshopId: UUID, slot: TimeRange) → bool
 - release(workshopId: UUID, slot: TimeRange)
-

2.6.3.2. Interface Layer

- **Controllers:** ServiceRequestController, MatchingController, OfferController, BookingController, ServiceOrderController, RescheduleController
- **Resources:**
 - SubmitServiceRequestResource { ownerId, vehicleId, serviceType, preferredWindow, location }

- OfferResource { offerId, serviceRequestId, workshopId, proposedSlot, price, status }
 - BookingResource { bookingId, serviceRequestId, offerId, workshopId, vehicleId, slot, status, createdAt }
 - RescheduleRequestResource { rescheduleRequestId, bookingId, requestedSlot, reason, status }
 - ServiceOrderResource { serviceOrderId, bookingId, workshopId, vehicleId, scheduledSlot, status, openedAt }
 - **Assemblers:** ServiceRequestAssembler, OfferAssembler, BookingAssembler, ServiceOrderAssembler, RescheduleAssembler
-

2.6.3.3. Application Layer

Aquí se implementan las integraciones con otros bounded contexts (ACL) y los servicios concretos.

- **Command Services:**
 - ServiceRequestCommandService
 - OfferCommandService
 - BookingCommandService
 - RescheduleCommandService
 - ServiceOrderCommandService
 - **Query Services:**
 - ServiceRequestQueryService
 - OfferQueryService
 - BookingQueryService
 - ServiceOrderQueryService
 - **Event Handlers:**
 - OnOfferAccepted → crea Booking y emite BookingCreatedEvent
 - OnBookingConfirmed → usa ScheduleAllocator (ACL) para reservar slot y emite ServiceOrderOpenedEvent
 - OnBookingCancelled → usa ScheduleAllocator (ACL) para liberar slot; puede emitir ServiceOrderClosedEvent
 - OnWorkshopProjectionUpdated (evento externo proyectado) → ajusta sugerencias de Matching
 - **ACL Implementations (implementaciones concretas de las interfaces del Domain Layer):**
 - WorkshopCalendarAclAvailabilityService (implementa AvailabilityService)
 - WorkshopCalendarAclScheduleAllocator (implementa ScheduleAllocator)
 - WorkshopPricingAcl (implementa PricingService)
 - PaymentsAcl (si BookingPolicy depende de pago)
 - IdentityAcl, NotificationsAcl
 - TrustingReputationAcl (dispara invitación a calificar tras cierre)
 - **Outbound Services:**
 - DomainEventPublisher
 - BookingNotificationSender
-

2.6.3.4. Infrastructure Layer

- **Repositories (implementaciones):**
 - ServiceRequestRepository : IServiceRequestRepository
 - OfferRepository : IOfferRepository
 - BookingRepository : IBookingRepository
 - ServiceOrderRepository : IServiceOrderRepository
- **External Services Adapters (utilizados por la Application Layer/ACL):**
 - WorkshopCalendarAdapter (conecta con Workshop BC)

- PaymentGatewayAdapter
 - MessageBrokerPublisher
 - NotificationAdapter
-

2.6.3.5 Bounded Context Software Architecture Component Level Diagrams

 Component-Vehicle-Maintenance

2.6.3.6 Bounded Context Software Architecture Code Level Diagrams

2.6.3.6.1 Bounded Context Domain Layer Class Diagrams

 Class-Vehicle

2.6.3.6.2 Bounded Context Database Design Diagram

 Database-Vehicle

2.6.4. Bounded Context: Workshop

2.6.4.1. Domain Layer

Este bounded context cubre la **gestión integral de los talleres (workshops)**: identidad, catálogo de servicios, disponibilidad operativa y personal técnico.

Su rol es mantener la información canónica del taller y exponerla a otros bounded contexts como Matching&Booking (para reservas) y Trusting&Reputation (para proyecciones de reputación).

No ejecuta procesos de matching, booking ni reputación, pero **publica eventos** que permiten integraciones.

Aggregates

- **Workshop**

Propósito: Entidad raíz que representa un taller registrado en la plataforma.

Atributos:

- workshopId: UUID
- legalName: string
- tradeName: string
- taxId: string
- status: WorkshopStatus (Active, Suspended, Deactivated)
- contacts: List
- addresses: List
- certifications: List
- operatingPolicy: OperatingPolicy
- capabilities: Capabilities
- services: List

Métodos:

- amendInformation(delta: WorkshopDelta)
- addService(service: WorkshopService)
- removeService(serviceId: UUID)
- setCapabilities(c: Capabilities)

- activate() / suspend(reason) / deactivate(reason)

Relaciones:

- 1 → * con WorkshopService
- 1 → 1 con WorkshopSchedule

- **WorkshopSchedule**

Propósito: Calendario operativo del taller, con reglas de disponibilidad y excepciones.

Atributos:

- workshopId: UUID
- businessHours: List
- exceptions: List
- timezone: string

Métodos:

- updateBusinessHours(windows: List)
- addException(window: ExceptionWindow)
- proposeSlots(constraints: SlotConstraints) → List

Relaciones:

- 1 → * con AvailabilitySlot

Entities

- **Mechanic**

Propósito: Representa un técnico/mecánico asociado al taller.

Atributos:

- mechanicId: UUID
- fullName: string
- skills: List
- certifications: List
- active: bool

Métodos:

- assignSkill(skill: Skill)
- revokeSkill(skill: Skill)

- **WorkshopService**

Propósito: Define un servicio que el taller ofrece a los clientes.

Atributos:

- serviceId: UUID
- serviceType: ServiceType
- stdDuration: Duration
- basePrice: Money
- requirements: List
- active: bool

Métodos:

- changePrice(newPrice: Money)
- activate() / deactivate()

Value Objects

- **Contact** – tipo (Phone, Email, WhatsApp) y valor.
- **Address** – line1, city, region, postalCode, country.
- **GeoPoint** – latitud, longitud.

- **Certification** – código, entidad emisora, vigencia.
 - **OperatingPolicy** – políticas de operación (cancelación, leadTime, etc.).
 - **Capabilities** – marcas, tipos de vehículo, equipamiento, capacidad máxima concurrente.
 - **Equipment** – nombre del equipo, servicios para los que aplica.
 - **AvailabilitySlot** – slotId, start, end, state (Free, Blocked, Unavailable), reason.
 - **BusinessWindow** – dayOfWeek, openTime, closeTime.
 - **ExceptionWindow** – start, end, reason.
-

Events (Domain Events)

- WorkshopRegisteredEvent(workshopId, occurredAt)
Propósito: Se registró un nuevo taller en la plataforma.
 - WorkshopInformationAmendedEvent(workshopId, delta, occurredAt)
Propósito: Se modificó la información del taller.
 - WorkshopScheduleUpdatedEvent(workshopId, occurredAt)
Propósito: Se actualizó el calendario operativo.
 - WorkshopAvailabilityBlockedEvent(workshopId, slotId, occurredAt)
Propósito: Un slot quedó bloqueado (ej. por booking confirmado).
 - WorkshopAvailabilityReleasedEvent(workshopId, slotId, occurredAt)
Propósito: Un slot bloqueado fue liberado.
 - MechanicAssignedEvent(workshopId, mechanicId, occurredAt)
Propósito: Se asignó un mecánico al taller.
 - MechanicUnassignedEvent(workshopId, mechanicId, occurredAt)
Propósito: Se desasignó un mecánico.
 - WorkshopDeactivatedEvent(workshopId, occurredAt)
Propósito: El taller quedó inactivo.
-

Services (Domain Interfaces)

- **ScheduleService**

Métodos:

- isAvailable(workshopId: UUID, slot: TimeRange) → bool
 - reserveSlot(workshopId: UUID, slot: TimeRange)
 - releaseSlot(workshopId: UUID, slot: TimeRange)
- Propósito:** Gestionar disponibilidad del calendario de talleres.

- **WorkshopPolicy**

Métodos:

- canActivate(workshopId: UUID) → bool
 - canAssignMechanic(workshopId: UUID, mechanicId: UUID) → bool
- Propósito:** Validar reglas de negocio específicas de talleres.
-

2.6.4.2. Interface Layer

Capa de exposición de APIs y DTOs para integración con otros contexts.

- **Controllers:** WorkshopController, ScheduleController, MechanicController.
- **Resources:**
 - WorkshopResource { workshopId, legalName, status, services, capabilities }
 - WorkshopScheduleResource { workshopId, businessHours, exceptions }
 - MechanicResource { mechanicId, fullName, skills, active }
 - WorkshopServiceResource { serviceId, serviceType, stdDuration, basePrice, active }

- **Assemblers:** WorkshopAssembler, ScheduleAssembler, MechanicAssembler, ServiceAssembler.
-

2.6.4.3. Application Layer

Implementa los casos de uso del negocio y la orquestación con otros contexts.

- **Command Services:** WorkshopCommandService, ScheduleCommandService, MechanicCommandService.
 - **Query Services:** WorkshopQueryService, ScheduleQueryService.
 - **Event Handlers:**
 - OnBookingConfirmed → bloquea slot (ScheduleService).
 - OnBookingCancelled → libera slot.
 - OnServiceOrderClosed → libera slot.
 - OnReviewSubmitted (Trusting&Reputation) → actualiza proyección de reputación.
 - **ACLs / Adaptadores:**
 - MatchingAcl (exposición de servicios y disponibilidad hacia Matching&Booking).
 - TrustingReputationAcl (publica información de talleres hacia Trusting&Reputation).
-

2.6.4.4. Infrastructure Layer

Implementaciones de persistencia y adaptadores externos.

- **Repositories:**
 - WorkshopRepository : IWorkshopRepository
 - ScheduleRepository : IScheduleRepository
 - MechanicRepository : IMechanicRepository
 - **External Adapters:**
 - ExternalCertificationAdapter (validación de certificaciones)
 - MessageBrokerPublisher (publicación de eventos a otros BCs)
-

2.6.4.5 Bounded Context Software Architecture Component Level Diagrams

 Component-Matching-Booking

2.6.4.6 Bounded Context Software Architecture Code Level Diagrams

2.6.4.6.1 Bounded Context Domain Layer Class Diagrams

 Class-Matching

2.6.4.6.2 Bounded Context Database Design Diagram

 Database-Matching

2.6.5. Bounded Context: Payment

2.6.5.1. Domain Layer

Este bounded context cubre la **gestión de suscripciones a la plataforma**, incluyendo la activación, renovación, cancelación y facturación asociada.

No gestiona pagos directos entre propietarios y talleres. Su alcance se limita a planes de suscripción de los distintos actores (owners, workshops, administradores).

Aggregates

- **Subscription**

Propósito: Representa una suscripción activa en la plataforma.

Atributos:

- subscriptionId: UUID
- subscriberId: UUID (puede ser ownerId o workshopId)
- planId: UUID
- status: SubscriptionStatus (Active, Suspended, Cancelled, Expired)
- startDate: DateTime
- endDate: DateTime
- renewalDate: DateTime
- paymentMethod: PaymentMethod

Métodos:

- activate(planId, paymentMethod)
- renew()
- cancel(reason: string)
- suspend(reason: string)
- expire()

Relaciones:

- 1 → 1 con Plan

- **Invoice**

Propósito: Documento de facturación generado en cada ciclo de suscripción.

Atributos:

- invoiceId: UUID
- subscriptionId: UUID
- amount: Money
- periodStart: DateTime
- periodEnd: DateTime
- issuedAt: DateTime
- status: InvoiceStatus (Issued, Paid, Failed)

Métodos:

- markPaid()
 - markFailed(reason: string)
-

Entities

- **Plan**

Propósito: Define las características de un plan de suscripción.

Atributos:

- planId: UUID
- name: string
- description: string
- price: Money
- duration: Duration (ej. mensual, anual)
- features: List

Métodos:

- updateDetails(name, description, price, duration, features)

Value Objects

- **Money** – amount, currency.
 - **PaymentMethod** – tipo (Card, PayPal, Transfer), tokenizado.
 - **Feature** – nombre, descripción.
-

Events (Domain Events)

- SubscriptionActivatedEvent(subscriptionId, subscriberId, planId, occurredAt)
Propósito: Se activó una nueva suscripción.
 - SubscriptionRenewedEvent(subscriptionId, planId, renewalDate, occurredAt)
Propósito: Se renovó una suscripción existente.
 - SubscriptionCancelledEvent(subscriptionId, reason, occurredAt)
Propósito: Una suscripción fue cancelada.
 - SubscriptionSuspendedEvent(subscriptionId, reason, occurredAt)
Propósito: La suscripción fue suspendida por incumplimiento o fallo de pago.
 - SubscriptionExpiredEvent(subscriptionId, occurredAt)
Propósito: La suscripción llegó a su fin y no fue renovada.
 - InvoiceIssuedEvent(invoiceId, subscriptionId, amount, occurredAt)
Propósito: Se generó una factura.
 - InvoicePaidEvent(invoiceId, occurredAt)
Propósito: Una factura fue pagada correctamente.
 - InvoiceFailedEvent(invoiceId, reason, occurredAt)
Propósito: Falló el cobro de la factura.
-

Services (Domain Interfaces)

- **BillingService**

Métodos:

- generateInvoice(subscriptionId: UUID) → Invoice
 - charge(invoiceId: UUID, paymentMethod: PaymentMethod) → bool
- Propósito:** Gestiona la facturación y cobro de suscripciones.

- **SubscriptionPolicy**

Métodos:

- canRenew(subscriptionId: UUID) → bool
 - canCancel(subscriptionId: UUID) → bool
- Propósito:** Define reglas de negocio sobre renovación y cancelación.
-

2.6.5.2. Interface Layer

Expone endpoints REST/GraphQL para gestión de suscripciones.

- **Controllers:** SubscriptionController, BillingController
 - **Resources:**
 - SubscriptionResource { subscriptionId, subscriberId, planId, status, startDate, endDate, renewalDate }
 - InvoiceResource { invoiceId, subscriptionId, amount, periodStart, periodEnd, status }
 - PlanResource { planId, name, description, price, duration, features }
 - **Assemblers:** SubscriptionAssembler, InvoiceAssembler, PlanAssembler
-

2.6.5.3. Application Layer

- **Command Services:**
 - SubscriptionCommandService
 - BillingCommandService
 - **Query Services:**
 - SubscriptionQueryService
 - InvoiceQueryService
 - PlanQueryService
 - **Event Handlers:**
 - OnInvoicePaid → renueva suscripción si aplica.
 - OnInvoiceFailed → suspende suscripción.
 - OnSubscriptionExpired → genera evento para notificar al usuario.
 - **ACLs:**
 - ExternalPaymentGatewayAcl (integra con proveedor de pagos externo).
-

2.6.5.4. Infrastructure Layer

Implementaciones de repositorios y adaptadores externos.

- **Repositories:**
 - SubscriptionRepository : ISubscriptionRepository
 - InvoiceRepository : IInvoiceRepository
 - PlanRepository : IPlanRepository
 - **External Adapters:**
 - PaymentGatewayAdapter (Stripe/PayPal/etc.)
 - MessageBrokerPublisher (publicación de eventos a otros BCs)
-

2.6.5.5 Bounded Context Software Architecture Component Level Diagrams

 Component-Trust

2.6.5.6 Bounded Context Software Architecture Code Level Diagrams

2.6.5.6.1 Bounded Context Domain Layer Class Diagrams

 Class-trusting

2.6.5.6.2 Bounded Context Database Design Diagram

 Database-trusting

2.6.6. Bounded Context: Identity & Access Management (IAM)

2.6.6.1. Domain Layer

Este bounded context cubre la gestión de identidades y accesos: registro, autenticación, autorización basada en roles y permisos, verificación de correo electrónico y recuperación de credenciales. Es transversal y provee mecanismos de seguridad para el resto de bounded contexts.

Aggregates

- **UserAccount**

Propósito: Representa la cuenta de un usuario.

Atributos:

- userId: UUID
- email: Email
- passwordHash: PasswordHash
- status: UserStatus (Active, Inactive, Locked, Deleted, PendingVerification)
- roles: List
- createdAt: DateTime
- lastLoginAt: DateTime?

Métodos:

- register(email: Email, passwordHash: PasswordHash)
- authenticate(candidateHash: PasswordHash)
- changePassword(newHash: PasswordHash)
- assignRole(role: Role) / revokeRole(role: Role)
- lock(reason: string) / deactivate() / activate()
- markEmailVerified()

Relaciones:

- 1 → * con RoleAssignment

- **Session**

Propósito: Gestiona una sesión autenticada y su token.

Atributos:

- sessionId: UUID
- userId: UUID
- token: Token
- issuedAt: DateTime
- expiresAt: DateTime
- status: SessionStatus (Active, Expired, Revoked)

Métodos:

- revoke()
 - refresh(newExpiresAt: DateTime)
-

Entities

- **Role**

Propósito: Agrupa permisos.

Atributos:

- roleId: UUID
- name: string
- permissions: List

- **RoleAssignment**

Propósito: Vincula un usuario con un rol.

Atributos:

- assignmentId: UUID
- userId: UUID
- roleId: UUID
- assignedAt: DateTime

- **Permission**

Propósito: Define acciones permitidas sobre recursos.

Atributos:

- permissionId: UUID
- name: string
- description: string

- **VerificationRequest**

Propósito: Gestiona el proceso de verificación de email.

Atributos:

- verificationId: UUID
- userId: UUID
- codeOrToken: string
- sentAt: DateTime
- expiresAt: DateTime
- status: VerificationStatus (Pending, Verified, Expired)

Métodos:

- verify(codeOrToken: string)
- expire()

- **RecoveryRequest**

Propósito: Gestiona la recuperación de contraseña.

Atributos:

- recoveryId: UUID
- userId: UUID
- codeOrToken: string
- requestedAt: DateTime
- expiresAt: DateTime
- status: RecoveryStatus (Pending, Completed, Expired, Revoked)

Métodos:

- complete(newHash: PasswordHash)
- expire()
- revoke(reason: string)

Value Objects

- **Email** – valida formato y unicidad.
- **PasswordHash** – encapsula el hash y su algoritmo.
- **Token** – valor opaco emitido para sesiones.
- **OtpCode** – valor de un solo uso con longitud y política de expiración.

Events (Domain Events)

- UserRegisteredEvent(userId, email, occurredAt)
Propósito: Se registró un usuario y quedó pendiente de verificación si aplica.

- EmailVerificationRequestedEvent(userId, verificationId, occurredAt)
Propósito: Se generó una solicitud de verificación y se envió el código o enlace.
- EmailVerifiedEvent(userId, verificationId, occurredAt)
Propósito: El correo fue verificado y la cuenta queda habilitada según política.
- EmailVerificationExpiredEvent(verificationId, occurredAt)
Propósito: La solicitud de verificación caducó.
- UserAuthenticatedEvent(userId, sessionId, occurredAt)
Propósito: Inicio de sesión exitoso.
- UserAuthenticationFailedEvent(email, reason, occurredAt)
Propósito: Intento fallido de autenticación.
- SessionStartedEvent(sessionId, userId, occurredAt)
Propósito: Se emitió un token de sesión.
- SessionRevokedEvent(sessionId, userId, occurredAt)
Propósito: Se revocó la sesión.
- PasswordResetRequestedEvent(userId, recoveryId, occurredAt)
Propósito: Se solicitó recuperación de contraseña y se envió código o enlace.
- PasswordResetCompletedEvent(userId, recoveryId, occurredAt)
Propósito: Se cambió la contraseña correctamente.
- PasswordResetExpiredEvent(recoveryId, occurredAt)
Propósito: La solicitud de recuperación expiró sin completarse.
- UserPasswordChangedEvent(userId, occurredAt)
Propósito: Se actualizó la contraseña desde una sesión autenticada.
- UserLockedEvent(userId, reason, occurredAt)
Propósito: La cuenta fue bloqueada por política.
- RoleAssignedEvent(userId, roleId, occurredAt) / RoleRevokedEvent(userId, roleId, occurredAt)
Propósito: Cambió la matriz de permisos efectivos.

Services (Domain Interfaces)

- **AuthenticationService**
Métodos: authenticate(email, password) → Token; refreshToken(token) → Token; revokeToken(token).
Propósito: Autenticación y ciclo de vida de sesiones.
- **AuthorizationService**
Métodos: hasPermission(userId, permission) → bool; getRoles(userId) → List.
Propósito: Evaluación de permisos basada en roles.
- **EmailVerificationService**
Métodos: request(userId) → VerificationRequest; verify(verificationId, codeOrToken) → bool.
Propósito: Orquesta el flujo de verificación de correo.
- **CredentialRecoveryService**
Métodos: request(userIdOrEmail) → RecoveryRequest; reset(recoveryId, codeOrToken, newHash) → bool.
Propósito: Orquesta el flujo de recuperación de contraseña.
- **PasswordPolicy**
Métodos: validate(rawPassword) → bool.
Propósito: Reglas de complejidad y caducidad.
- **OtpPolicy**
Métodos: generate() → OtpCode; validate(codeOrToken, scope) → bool; ttl(scope) → Duration.
Propósito: Política de códigos y tokens de un solo uso.

Commands (visión global)

Son records inmutables que expresan intención de cambio. Activan métodos de agregados, validan políticas y emiten eventos.

Ejemplos:

- RegisterUser(email: string, password: string)
 - AuthenticateUser(email: string, password: string)
 - StartSession(userId: UUID)
 - RevokeSession(sessionId: UUID)
 - RequestEmailVerification(userId: UUID)
 - VerifyEmail(verificationId: UUID, codeOrToken: string)
 - RequestPasswordReset(userIdOrEmail: string)
 - CompletePasswordReset(recoveryId: UUID, codeOrToken: string, newPassword: string)
 - ChangePassword(userId: UUID, currentPassword: string, newPassword: string)
 - AssignRole(userId: UUID, roleId: UUID) / RevokeRole(userId: UUID, roleId: UUID)
 - LockUser(userId: UUID, reason: string) / ActivateUser(userId: UUID)
-

Queries (visión global)

Consultas inmutables orientadas a lectura.

Ejemplos:

- GetUserById(userId: UUID)
 - GetUserByEmail(email: string)
 - GetUserRoles(userId: UUID)
 - GetActiveSessions(userId: UUID)
 - GetVerificationStatus(userId: UUID)
 - GetRecoveryStatus(recoveryId: UUID)
-

2.6.6.2. Interface Layer

Capa de exposición de APIs y DTOs.

- **Controllers:** AuthController, UserController, RoleController, VerificationController, RecoveryController
 - **Resources:**
 - UserResource { userId, email, status, roles, createdAt, lastLoginAt }
 - SessionResource { sessionId, userId, token, expiresAt, status }
 - RoleResource { roleId, name, permissions }
 - VerificationResource { verificationId, userId, status, expiresAt }
 - RecoveryResource { recoveryId, userId, status, expiresAt }
 - **Assemblers:** UserAssembler, SessionAssembler, RoleAssembler, VerificationAssembler, RecoveryAssembler
-

2.6.6.3. Application Layer

Implementa casos de uso y orquestación con servicios externos.

- **Command Services:**
 - UserCommandService
 - SessionCommandService
 - RoleCommandService
 - VerificationCommandService
 - RecoveryCommandService
- **Query Services:**

- UserQueryService
- RoleQueryService
- SessionQueryService
- VerificationQueryService
- RecoveryQueryService

- **Event Handlers:**

- OnUserRegistered → crea VerificationRequest y dispara EmailVerificationRequestedEvent.
- OnEmailVerified → cambia estado de usuario a Active o mantiene según política.
- OnPasswordResetRequested → genera RecoveryRequest y envía notificación.
- OnPasswordResetCompleted → invalida sesiones previas y registra UserPasswordChangedEvent.
- OnUserPasswordChanged → revoca sesiones activas excepto la actual según política.
- OnAuthenticationFailedThresholdReached → bloquea cuenta y emite UserLockedEvent.

- **ACLs / Integraciones:**

- NotificationAcl (envío de emails/SMS para verificación y recuperación).
- ExternalIdProviderAcl (SSO/OAuth).
- TokenProviderAcl (emisión/validación de JWT o similar).
- AuditTrailAcl (registro de auditoría de seguridad).

2.6.6.4. Infrastructure Layer

Implementaciones concretas de repositorios y adaptadores.

- **Repositories:**

- UserRepository : IUserRepository
- SessionRepository : ISessionRepository
- RoleRepository : IRoleRepository
- VerificationRepository : IVerificationRepository
- RecoveryRepository : IRecoveryRepository

- **External Adapters:**

- EmailProviderAdapter
- SmsProviderAdapter
- TokenProviderAdapter
- MessageBrokerPublisher
- AuditLogAdapter

2.6.6.5 Bounded Context Software Architecture Component Level Diagrams

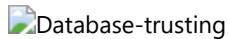
 Component-Suscription

2.6.6.6 Bounded Context Software Architecture Code Level Diagrams

2.6.6.6.1 Bounded Context Domain Layer Class Diagrams

 Class-payment

2.6.6.6.2 Bounded Context Database Design Diagram



2.6.7. Bounded Context: Notifications

2.6.7.1. Domain Layer

Este bounded context gestiona la orquestación y entrega de notificaciones en tres canales: **Push, Email y SMS**.

Soporta mensajes transaccionales y mensajes programados No incorpora lógica de negocio de otros BCs; consume sus eventos y aplica plantillas, preferencias y políticas de envío.

Aggregates

- **Notification**

Propósito: Representa una notificación a uno o más destinatarios, con plantillas y rutas por canal.

Atributos:

- notificationId: UUID
- correlationId: string
- type: NotificationType (Transactional, Security, Reminder)
- recipients: List
- templateId: UUID
- locale: string
- payload: Map<string,string>
- channelPlan: List
- scheduledAt: DateTime?
- priority: Priority (High, Normal, Low)
- status: NotificationStatus (Pending, Scheduled, Sending, Sent, Failed, Cancelled)

Métodos:

- schedule(at: DateTime)
- send()
- cancel(reason: string)
- recordAttempt(result: DeliveryResult)

- **Template**

Propósito: Contenido parametrizable por **canal e idioma**.

Atributos:

- templateId: UUID
- key: string (ej. booking.confirmed)
- version: int
- channels: Map<Channel, ChannelTemplate>
- variables: List
- defaultLocale: string

Métodos:

- render(channel: Channel, locale: string, payload: Map) → RenderedMessage
- newVersion(changes)

Entities

- **DeliveryAttempt**

Propósito: Trazabilidad por canal/proveedor.

Atributos:

- attemptId: UUID
- notificationId: UUID
- channel: Channel (Email, Push, SMS)
- provider: string
- requestedAt: DateTime
- status: AttemptStatus (Queued, Sent, Delivered, Opened, Clicked, Bounced, Failed)
- error?: string

Métodos:

- markDelivered()/markOpened()/markClicked(url)
- markBounced(reason)/markFailed(reason)

- **Preference**

Propósito: Preferencias por usuario y por tipo/canal.

Atributos:

- preferenceId: UUID
- userId: UUID
- locale: string
- channelOptIn: Map<Channel,bool>
- typeOptIn: Map<NotificationType,bool>
- quietHours?: TimeRange

Métodos:

- setOptIn(scope, enabled: bool)
- setQuietHours(range: TimeRange)

Value Objects

- **ChannelRoute** – channel, primaryProvider, fallbackProviders[], retries, timeoutMs
- **RenderedMessage** – subject?, body, richBody?, data?
- **VariableSpec** – name, required, description
- **DeliveryResult** – status, providerMessageId?, error?
- **TimeRange** – startTime, endTime

Events (Domain Events)

- NotificationRequestedEvent(notificationId, type, occurredAt)
Propósito: Se solicitó una notificación (por evento externo o API).
- NotificationScheduledEvent(notificationId, scheduledAt, occurredAt)
Propósito: Quedó programada para envío futuro.
- NotificationSentEvent(notificationId, occurredAt)
Propósito: Todos los intentos fueron despachados.
- DeliveryStatusUpdatedEvent(notificationId, attemptId, channel, status, occurredAt)
Propósito: Actualización de estado por canal/proveedor.
- NotificationFailedEvent(notificationId, reason, occurredAt)
Propósito: Falló definitivamente tras reintentos.
- PreferenceChangedEvent(userId, occurredAt)
Propósito: Cambiaron las preferencias y afectan el enrutamiento.

Services (Domain Interfaces)

- **TemplateRenderer**

- render(templateId, channel, locale, payload) → RenderedMessage

Propósito: Resuelve versión/idioma y genera el contenido final.

- **ChannelRouter**

- route(notification: Notification) → List
- dispatch(route, message) → DeliveryResult

Propósito: Selección de canal/proveedor, reintentos y failover.

- **PreferenceService**

- check(userId, type, channel, at: DateTime) → bool

Propósito: Respeta opt-in/out y quiet hours.

- **SchedulerService**

- enqueue(notificationId, at: DateTime)

Propósito: Programa recordatorios y ventanas de envío.

2.6.7.2. Interface Layer

- **Controllers:** NotificationController, TemplateController, PreferenceController

- **Resources:**

- NotificationResource { notificationId, type, recipients, templateId, scheduledAt, status }
- TemplateResource { templateId, key, version, channels, variables }
- PreferenceResource { userId, channelOptIn, typeOptIn, quietHours }

- **Assemblers:** NotificationAssembler, TemplateAssembler, PreferenceAssembler

2.6.7.3. Application Layer

- **Command Services:** NotificationCommandService, TemplateCommandService, PreferenceCommandService

- **Query Services:** NotificationQueryService, TemplateQueryService

- **Event Handlers (ejemplos):**

- OnBookingConfirmed (Matching&Booking) → NotificationRequestedEvent (Transactional)
- OnInvoiceIssued (Payment) → NotificationRequestedEvent (Transactional)
- ScheduledReminderTick → envía recordatorios de mantenimiento (Reminder)

- **ACLs:**

- EmailProviderAcl, PushProviderAcl, SmsProviderAcl
- IAMAcl (para obtener correos/tokens push verificados)

2.6.7.4. Infrastructure Layer

- **Repositories:** NotificationRepository, TemplateRepository, PreferenceRepository, DeliveryAttemptRepository

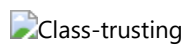
- **External Adapters:** EmailAdapter, PushAdapter, SmsAdapter, MessageBrokerPublisher, SchedulerAdapter

2.6.7.5 Bounded Context Software Architecture Component Level Diagrams

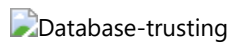
 Component-Notifications

2.6.7.6 Bounded Context Software Architecture Code Level Diagrams

2.6.7.6.1 Bounded Context Domain Layer Class Diagrams



2.6.7.6.2 Bounded Context Database Design Diagram





Conclusiones

Integración de metodologías: La combinación de Lean UX, User Stories y Domain-Driven Design (DDD) proporciona un marco sólido para diseñar soluciones centradas en el usuario y alineadas con los objetivos del negocio.

Importancia del lenguaje común: El uso de Ubiquitous Language asegura que tanto el equipo técnico como los usuarios comprendan los conceptos clave sin ambigüedades, reduciendo errores en el desarrollo.

Validación constante: Las entrevistas y herramientas como Empathy Mapping o User Personas permiten identificar necesidades reales y garantizar que la solución propuesta responda a problemas concretos.

Trazabilidad y claridad: El uso de User Stories, Impact Mapping y Product Backlog aporta organización y priorización en los requerimientos, facilitando la gestión ágil del proyecto.

Escalabilidad del diseño: EventStorming y los diagramas de arquitectura de software permiten visualizar la complejidad del sistema, anticipar dependencias y construir una solución escalable y mantenible.

Bibliografía

Fowler, M. (2004). Ubiquitous Language. Recuperado de: <https://martinfowler.com/bliki/UbiquitousLanguage.html>

Gothelf, J. (2016). Lean UX – Chapter 3. Scribd. Recuperado de: <https://www.scribd.com/document/655516553/Leanux-Sampler>

Cohn, M. (s. f.). User Stories Articles. Mountain Goat Software. Recuperado de: <https://www.mountaingoatsoftware.com/blog/tag/user-stories>

UXPressia. (2022). How to create an Impact Map in 4 easy steps? Recuperado de: <https://uxpressia.com/blog/build-impact-map-4-easy-steps>

Nielsen Norman Group. (2019). Empathy Mapping: The First Step in Design Thinking. Recuperado de: <https://www.nngroup.com/articles/empathy-mapping/>

DDD by Examples. (s. f.). Big Picture EventStorming. Recuperado de: <https://github.com/ddd-by-examples/library/blob/master/docs/bigpicture.md>

Tune, N. (2019). Domain-Driven Architecture Diagrams. Medium. Recuperado de: <https://medium.com/nick-tune-tech-strategy-blog/domain-drivenarchitecture-diagrams-139a75acb578>
